

Estadística I

[GUÍA RÁPIDA DE ESTUDIO]

@garydevelop

GM

Índice

Parte 1 · Tratamiento estadístico de datos	4	Parte 4 · Variables aleatorias continuas	30
1 Qué es la estadística	4	36 Variable aleatoria continua y función de densidad	30
2 Tipos de variables y escalas de medición	5	37 Función de distribución acumulada continua	30
3 Tablas de frecuencias	5	38 Esperanza y varianza continuas	31
4 Datos agrupados en intervalos	6	39 Percentiles y cuantiles	32
5 Gráficos estadísticos	7	40 Distribución uniforme continua	32
6 Medidas de tendencia central	8	41 Distribución exponencial	33
7 Medidas de posición	9	42 Distribución gamma	34
8 Medidas de dispersión	9	43 Distribución normal	34
9 Asimetría y curtosis	10	44 Normal estándar y estandarización	35
10 Diagrama de caja y datos atípicos	11	45 Aproximación normal a la binomial	36
11 Correlación entre dos variables	12	46 Distribución Weibull	37
Parte 2 · Métodos de enumeración y probabilidad	13	47 Cómo elegir el modelo continuo	38
12 Experimento aleatorio, espacio muestral y eventos	13	Parte 5 · Vectores aleatorios	39
13 Operaciones con eventos	13	48 Vector aleatorio y distribución conjunta discreta	39
14 Principios de conteo	14	49 Distribuciones marginales	40
15 Factorial y permutaciones	15	50 Distribuciones condicionales	40
16 Combinaciones	15	51 Independencia de variables aleatorias	41
17 Definición de probabilidad	16	52 Esperanza de funciones de X e Y	42
18 Regla de la suma de probabilidades	17	53 Covarianza	43
19 Probabilidad condicional y regla del producto	17	54 Correlación	44
20 Independencia de eventos	18	55 Combinaciones lineales	45
21 Probabilidad total	18	56 Distribución conjunta continua	45
22 Teorema de Bayes	19	57 Distribución multinomial	47
Parte 3 · Variables aleatorias discretas	20	58 Distribución normal bivariada	47
23 Variable aleatoria discreta y función de masa	20	Parte 6 · Convergencia y distribuciones muestrales	49
24 Función de distribución acumulada	21	59 Muestra aleatoria y estadístico	49
25 Esperanza o valor esperado	21	60 Distribución muestral de la media	50
26 Varianza y desviación estándar	22	61 Ley de los grandes números	51
27 Distribución uniforme discreta	23	62 Teorema del Límite Central	52
28 Distribución de Bernoulli	23	63 Convergencia en distribución	53
29 Distribución binomial	24	64 Distribución de la proporción muestral	54
30 Distribución geométrica	25	65 Distribución de la diferencia de medias	54
31 Distribución binomial negativa	26	66 Distribución chi-cuadrado	55
32 Distribución hipergeométrica	26	67 Distribución de la varianza muestral	56
33 Distribución de Poisson	27	68 Distribución t de Student	57
34 Aproximación de Poisson a la binomial	28	69 Distribución F de Fisher	58
35 Cómo elegir el modelo	29	70 Estimadores y sesgo	59
		71 Intervalo de confianza para la media	59

Cómo usar esta guía

Una guía rápida de **Estadística I**: conceptos cortos, trucos para recordarlos y tablas con fórmulas y ejemplos con resultados calculados de verdad.

1 · Lee el concepto

Dos líneas que explican de qué trata el tema, sin vueltas.

2 · Memoriza el truco

La caja **RECUERDA** trae una comparación fácil para no olvidar la idea.

3 · Prueba la tabla

Cada fila trae **medida, descripción, ejemplo y resultado**. Cópialo y córrelo tú mismo.

Cómo leer una tabla

MEDIDA	DESCRIPCIÓN	EJEMPLO	RESULTADO
media	Qué es y su fórmula, en una frase.	<code>np.mean([6, 8, 10])</code>	8.0

Sobre el curso

- Estadística I es una materia obligatoria de formación básica (**5.8 ECTS**): 3 horas de docencia, 1 de práctica y 5 de trabajo autónomo por semana. Se evalúa con exámenes, lecciones, tareas, proyectos y participación.
- El objetivo es interpretar datos, graficarlos, usar probabilidad con modelos discretos y continuos, y entender el Teorema del Límite Central como base de la inferencia. La guía sigue el orden del sílabo; el texto base es *Probabilidad y Estadística: Fundamentos y Aplicaciones*, de G. Zurita.

Mapa de la guía

PARTE 1 Tratamiento estadístico de datos **pág. 4**

Cómo ordenar, resumir y describir un conjunto de datos. (11 temas · 6 h en el sílabo)

PARTE 2 Métodos de enumeración y probabilidad **pág. 13**

Contar posibilidades y medir la incertidumbre con probabilidades. (11 temas · 9 h en el sílabo)

PARTE 3 Variables aleatorias discretas **pág. 20**

Modelos para contar: éxitos, fallas y eventos que ocurren. (13 temas · 6 h en el sílabo)

PARTE 4 Variables aleatorias continuas **pág. 30**

Modelos para medir: tiempos, alturas, pesos y otras cantidades continuas. (12 temas · 9 h en el sílabo)

PARTE 5 Vectores aleatorios **pág. 39**

Dos o más variables aleatorias al mismo tiempo y cómo se relacionan. (11 temas · 6 h en el sílabo)

PARTE 6 Convergencia y distribuciones muestrales **pág. 49**

Del azar de una muestra a las bases de la estadística inferencial. (13 temas · 6 h en el sílabo)

Antes de empezar

- Los resultados de las tablas salieron de **ejecutar el código de verdad** (Python 3.11, NumPy 2.4, Pandas 3.0, SciPy 1.17, Matplotlib 3.10) y están redondeados, casi siempre a 4 cifras.
- Los **datos de ejemplo** arriba de una tabla (por ejemplo `notas = [6, 7, 9]`) ya existen en todos los ejemplos del tema.
- SciPy nombra los parámetros a su manera (la exponencial usa `scale = 1/λ`). Cada tema lo aclara donde importa.
- En un examen se trabaja con calculadora y tablas: usa el código para **comprobar** lo que resuelves a mano.

Material de estudio no oficial, hecho por estudiantes para estudiar. No reemplaza las clases, el sílabo ni el material oficial de la materia.

Tratamiento estadístico de datos

Cómo ordenar, resumir y describir un conjunto de datos.

TEMA 1

Qué es la estadística

La estadística es la ciencia de recolectar, organizar, analizar e interpretar datos para tomar decisiones. La descriptiva resume lo que tienes; la inferencial usa una parte para concluir sobre el todo.

RECUERDA Probar una cucharada de sopa (muestra) para saber si toda la olla (población) tiene sal.

DATOS DE EJEMPLO

```
import numpy as np
poblacion = [5, 6, 6, 7, 7, 7, 8, 8, 9, 10]
muestra = poblacion[::2]
```

CONCEPTO	DESCRIPCIÓN	EJEMPLO	RESULTADO
Estadística descriptiva	Resume y presenta los datos que ya tienes.	<pre>print(min(poblacion), max(poblacion)) print(np.mean(poblacion))</pre>	5 10 7.3
Estadística inferencial	Con una muestra, concluye sobre la población. Aquí: error de estimación.	<pre>e = np.mean(muestra) - np.mean(poblacion) print(round(e, 2))</pre>	-0.3
Población (N)	Todos los elementos que se quieren estudiar.	<pre>print(len(poblacion))</pre>	10
Muestra (n)	Parte de la población que sí se observa.	<pre>print(muestra) print(len(muestra))</pre>	[5, 6, 7, 8, 9] 5
Parámetro	Número que describe a la población (μ , σ). Casi siempre es desconocido.	<pre>print(np.mean(poblacion))</pre>	7.3
Estadístico	Número calculado con la muestra (media muestral, s). Estima al parámetro.	<pre>print(np.mean(muestra))</pre>	7.0
Dato	Cada valor observado de un individuo.	<pre>print(poblacion[0])</pre>	5

Aquí la población son 10 notas de un curso y la muestra son 5 de ellas.

Tipos de variables y escalas de medición

RECUERDA Nominal = nombres, ordinal = ranking, intervalo = termómetro, razón = balanza (el cero sí significa nada).

```
import pandas as pd
df = pd.DataFrame({
    "carrera": ["Computación", "Química", "Computación", "Civil"],
    "hermanos": [0, 2, 1, 3],
    "altura": [1.62, 1.75, 1.68, 1.80]})
```

TEMA 3

Tablas de frecuencias

RECUERDA Es como el conteo de votos: primero cuántos votos tiene cada candidato, luego qué parte del total son.

```
import pandas as pd
x = pd.Series(["A", "B", "A", "C", "B", "A", "A", "C", "B", "A"])
f = x.value_counts().sort_index()
```

GM Estadística I · @garydevelop

FRECUENCIA	DESCRIPCIÓN	EJEMPLO	RESULTADO																
Relativa hi	hi = fi / n. La suma de todas da 1.	<code>print((f / len(x)).round(2))</code>	A 0.5 B 0.3 C 0.2 Name: count, dtype: float64																
Porcentual pi	pi = hi · 100.	<code>print((f / len(x) * 100).round(1))</code>	A 50.0 B 30.0 C 20.0 Name: count, dtype: float64																
Acumulada Fi	Suma de las fi hasta ese valor: <code>cumsum()</code> .	<code>print(f.cumsum())</code>	A 5 B 8 C 10 Name: count, dtype: int64																
Relativa acumulada Hi	Fi / n. El último valor siempre es 1.	<code>print((f.cumsum() / len(x)).round(2))</code>	A 0.5 B 0.8 C 1.0 Name: count, dtype: float64																
Tabla completa	Todo junto en un DataFrame.	<code>t = pd.DataFrame({"fi": f}) t["hi"] = (f / len(x)).round(2) t["Fi"] = f.cumsum() print(t)</code>	<table> <tr><th></th><th>fi</th><th>hi</th><th>Fi</th></tr> <tr><td>A</td><td>5</td><td>0.5</td><td>5</td></tr> <tr><td>B</td><td>3</td><td>0.3</td><td>8</td></tr> <tr><td>C</td><td>2</td><td>0.2</td><td>10</td></tr> </table>		fi	hi	Fi	A	5	0.5	5	B	3	0.3	8	C	2	0.2	10
	fi	hi	Fi																
A	5	0.5	5																
B	3	0.3	8																
C	2	0.2	10																

Los datos son el paralelo de 10 estudiantes. Para variables ordinales o numéricas, `sort_index()` ordena la tabla.

TEMA 4

Datos agrupados en intervalos

Cuando hay muchos valores distintos, se agrupan en clases (intervalos). Se calcula el rango, el número de clases con la regla de Sturges y el ancho de cada clase.

RECUERDA Es como ordenar ropa por tallas (S, M, L) en vez de por centímetros exactos.

DATOS DE EJEMPLO

```
import math
import numpy as np
import pandas as pd
e = np.array([18, 19, 19, 20, 20, 20, 21, 21, 22, 22,
              22, 23, 23, 24, 25, 25, 26, 28, 30, 35])
```

PASO	DESCRIPCIÓN	EJEMPLO	RESULTADO
Rango R	R = dato mayor – dato menor.	print(np.ptp(e))	17
Sturges (k)	k = 1 + 3.322 · log10(n) . Se redondea.	print(round(1 + 3.322 * math.log10(len(e)), 2))	5.32
Ancho de clase	c = R / k, redondeado hacia arriba.	print(math.ceil(np.ptp(e) / 5))	4
Límites de clase	Se parte del mínimo sumando el ancho.	b = np.arange(18, 39, 4) print(b)	[18 22 26 30 34 38]
pd.cut()	Asigna cada dato a su clase. [a, b) incluye a y excluye b.	b = np.arange(18, 39, 4) c = pd.cut(e, b, right=False) print(c.value_counts().sort_index())	[18, 22) 8 [22, 26) 8 [26, 30) 2 [30, 34) 1 [34, 38) 1 Name: count, dtype: int64
Marca de clase	Punto medio = (límite inf + límite sup) / 2.	b = np.arange(18, 39, 4) print((b[:-1] + b[1:]) / 2)	[20. 24. 28. 32. 36.]
Media agrupada	Σ (marca · fi) / n. Aproxima a la media real.	m = np.array([20, 24, 28, 32, 36]) f = np.array([8, 8, 2, 1, 1]) print((m * f).sum() / f.sum())	23.8

Los datos son las edades de 20 estudiantes. Con $n = 20$, Sturges da 5.32, es decir 5 clases.

TEMA 5

Gráficos estadísticos

Cada tipo de dato tiene su gráfico. Las variables cualitativas usan barras o circular. Las cuantitativas continuas usan histograma, polígono de frecuencias u ojiva.

RECUERDA Barras = categorías separadas. Histograma = barras pegadas porque el eje es continuo.

DATOS DE EJEMPLO

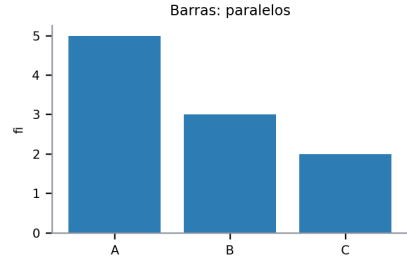
```
import numpy as np
import matplotlib.pyplot as plt
e = np.array([18, 19, 19, 20, 20, 20, 21, 21, 22, 22,
              22, 23, 23, 24, 25, 25, 26, 28, 30, 35])
```

GRÁFICO	DESCRIPCIÓN	EJEMPLO	RESULTADO
<code>plt.bar()</code>	Barras separadas, una por categoría.	<code>plt.bar(["A", "B", "C"], [5, 3, 2])</code>	Barra de 5, de 3 y de 2
<code>plt.pie()</code>	Circular: cada categoría es una parte del 100 %.	<code>plt.pie([5, 3, 2], labels=["A", "B", "C"])</code>	Pastel de 50 %, 30 % y 20 %
<code>plt.hist()</code>	Histograma: barras pegadas por intervalo.	<code>plt.hist(e, bins=[18, 22, 26, 30, 34, 38])</code>	Barras pegadas de 8, 8, 2, 1, 1
<code>np.histogram()</code>	Calcula las alturas sin dibujar.	<code>b = [18, 22, 26, 30, 34, 38]</code> <code>print(np.histogram(e, bins=b)[0])</code>	[8 8 2 1 1]
Polígono	Une las marcas de clase con la frecuencia.	<code>plt.plot([20, 24, 28, 32, 36], [8, 8, 2, 1, 1])</code>	Línea que sube y baja
Ojiva	Frecuencia acumulada contra el límite superior.	<code>plt.plot([18, 22, 26, 30, 34, 38], [0, 8, 16, 18, 19, 20])</code>	Curva que solo sube hasta $n = 20$
Cuál usar	Depende del tipo de variable.	Tipo de variable	Cualitativa: barras, circular Cuantitativa: histograma Acumulados: ojiva

Los gráficos de ejemplo están al final de este tema.

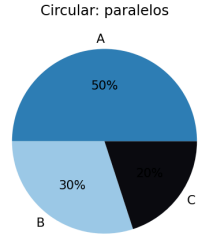
Gráficos de ejemplo

Barras: paralelos



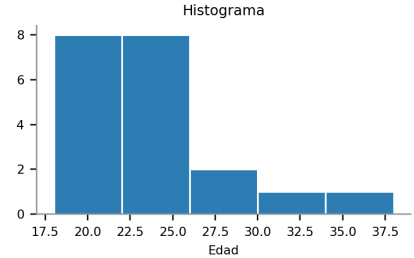
```
plt.bar(["A", "B", "C"], [5, 3, 2])
plt.ylabel("fi")
plt.title("Barras: paralelos")
plt.tight_layout()
```

Circular: paralelos

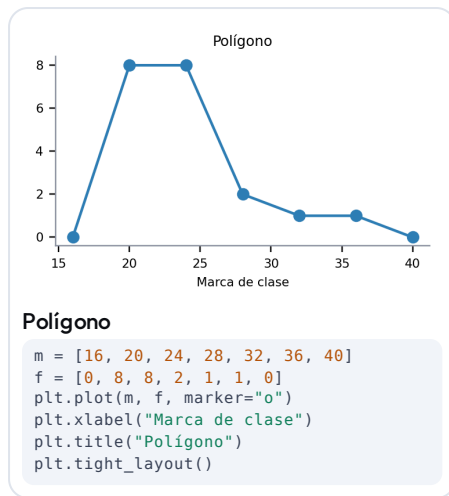


```
plt.pie([5, 3, 2], labels=["A", "B", "C"], autopct="%d%")
plt.title("Circular: paralelos")
plt.tight_layout()
```

Histograma



```
e = [18, 19, 19, 20, 20, 20, 21, 21, 22, 22, 22, 23, 23, 24, 25, 25, 26, 28, 30, 35]
plt.hist(e, bins=[18, 22, 26, 30, 34, 38], edgecolor="white")
plt.xlabel("Edad")
plt.title("Histograma")
plt.tight_layout()
```



TEMA 6

Medidas de tendencia central

Resumen en un solo número del centro de los datos. La media es el promedio, la mediana es el valor del medio y la moda es el que más se repite.

RECUERDA Media = repartir todo parejo. Mediana = la persona del medio de la fila. Moda = la más popular.

DATOS DE EJEMPLO

```
import numpy as np
import statistics
from scipy import stats
s = [450, 500, 500, 550, 600, 650, 3000]
```

MEDIDA	DESCRIPCIÓN	EJEMPLO	RESULTADO
Media	media = suma de datos / n. Muy sensible a atípicos.	<code>print(round(np.mean(s), 2))</code>	892.86
Mediana	Valor central de los datos ordenados.	<code>print(np.median(s))</code>	550.0
Moda	Valor que más se repite.	<code>print(statistics.mode(s))</code>	500
Moda múltiple	Si hay empate, hay varias modas (bimodal).	<code>print(statistics.multimode([1, 1, 2, 2, 3]))</code>	[1, 2]
Media ponderada	$\Sigma(\text{peso} \cdot \text{dato}) / \Sigma \text{ pesos}$. Cada dato pesa distinto.	<code>notas = [8, 6, 9]</code> <code>pesos = [0.3, 0.3, 0.4]</code> <code>print(round(np.average(notas, weights=pesos), 2))</code>	7.8
Media recortada	Descarta un % de cada extremo y promedia.	<code>print(stats.trim_mean(s, 0.2))</code>	560.0
Efecto del atípico	Sin el sueldo de 3000 la media baja mucho; la mediana casi no.	<code>print(round(np.mean(s[:-1]), 2))</code> <code>print(np.median(s[:-1]))</code>	541.67 525.0
Cuándo usar cada una	Depende de la forma de los datos.	Elegir	Simétricos: media Con atípicos: mediana Categorías: moda Pesos distintos: ponderada

Los datos son los sueldos (USD) de 7 empleados.

TEMA 7

Medidas de posición

Dividen los datos ordenados en partes iguales. Los cuartiles los parten en 4, los deciles en 10 y los percentiles en 100. El percentil k deja el k % de los datos por debajo.

RECUERDA Es como ver en qué lugar de la fila quedaste: percentil 90 significa que superas al 90 % del grupo.

DATOS DE EJEMPLO

```
import numpy as np
import pandas as pd
from scipy import stats
n = [4, 5, 6, 6, 7, 7, 8, 8, 9, 9, 10]
```

MEDIDA	DESCRIPCIÓN	EJEMPLO	RESULTADO
Cuartiles	Q1 = P25, Q2 = mediana = P50, Q3 = P75.	<code>print(np.percentile(n, [25, 50, 75]))</code>	[6. 7. 8.5]
Deciles	D1 a D9: parten en 10 partes iguales.	<code>print(np.percentile(n, range(10, 100, 10)))</code>	[5. 6. 6. 7. 7. 8. 8. 9. 9.]
Percentil	P90: bajo este valor está el 90 % de los datos.	<code>print(np.percentile(n, 90))</code>	9.0
Rango intercuartil	IQR = Q3 - Q1. Mide la dispersión del 50 % central.	<code>q1, q3 = np.percentile(n, [25, 75])</code> <code>print(q3 - q1)</code>	2.5
Posición	$\text{pos} = (n - 1) \cdot p$ y se interpola entre vecinos.	<code>print((len(n) - 1) * 0.75)</code>	7.5
Percentil de un valor	Porcentaje de datos que quedan por debajo de 8.	<code>print(round(stats.percentileofscore(n, 8), 2))</code>	68.18
Series.quantile()	Lo mismo en pandas, con p entre 0 y 1.	<code>print(pd.Series(n).quantile(0.75))</code>	8.5
describe()	Resumen con cuartiles incluidos.	<code>print(pd.Series(n).describe().round(2))</code>	count 11.00 mean 7.18 std 1.83 min 4.00 25% 6.00 50% 7.00 75% 8.50 max 10.00 dtype: float64

Los datos son las notas de 11 estudiantes.

TEMA 8

Medidas de dispersión

Dicen qué tan separados están los datos entre sí. Una media sola no basta: dos cursos pueden tener el mismo promedio y ser muy distintos por dentro.

RECUERDA Dos cursos con promedio 7: en uno todos sacan 7 y en otro unos sacan 2 y otros 10. La dispersión los distingue.

DATOS DE EJEMPLO

```
import numpy as np
import pandas as pd
x = [2, 4, 4, 4, 5, 5, 7, 9]
```

MEDIDA	DESCRIPCIÓN	EJEMPLO	RESULTADO
Rango	máximo - mínimo. Solo usa dos datos.	<code>print(np.ptp(x))</code>	7
Varianza poblacional	$\sigma^2 = \sum (\text{dato} - \mu)^2 / N$. Es ddof=0.	<code>print(np.var(x))</code>	4.0

MEDIDA	DESCRIPCIÓN	EJEMPLO	RESULTADO
Varianza muestral	$s^2 = \sum (\text{dato} - \text{media})^2 / (n - 1)$. Es ddof=1.	<code>print(round(np.var(x, ddof=1), 4))</code>	4.5714
Desviación poblacional	$\sigma = \sqrt{s^2}$. Misma unidad que los datos.	<code>print(np.std(x))</code>	2.0
Desviación muestral	$s = \sqrt{s^2}$. La que se usa con muestras.	<code>print(round(np.std(x, ddof=1), 4))</code>	2.1381
pandas vs numpy	<code>std()</code> de pandas usa ddof=1; el de numpy, ddof=0.	<code>print(round(pd.Series(x).std(), 4))</code> <code>print(np.std(x))</code>	2.1381 2.0
Coef. de variación	CV = $s / \text{media} \cdot 100$. Dispersión relativa en %.	<code>print(round(np.std(x, ddof=1) / np.mean(x) * 100, 2))</code>	42.76
CV para comparar	Compara variables con unidades distintas (altura y peso).	<code>a = np.array([160, 165, 170, 175, 180])</code> <code>p = np.array([55, 60, 70, 80, 90])</code> <code>def cv(v):</code> <code>return</code> <code>v.std(ddof=1)/v.mean()*100</code> <code>print(round(cv(a), 1),</code> <code>round(cv(p), 1))</code>	4.7 20.2

Los datos son las notas de 8 estudiantes (media 5). Usa $n - 1$ cuando trabajes con una muestra.

TEMA 9

Asimetría y curtosis

La asimetría dice hacia qué lado se estira la cola de la distribución. La curtosis dice qué tan puntiaguda o plana es respecto a una normal (que vale 0 en scipy).

RECUERDA Asimetría positiva = cola larga a la derecha, como los sueldos: pocos ganan muchísimo.

DATOS DE EJEMPLO

```
import numpy as np
from scipy import stats
sim = [1, 2, 3, 4, 5, 6, 7]
der = [450, 500, 500, 550, 600, 650, 3000]
izq = [2, 7, 8, 8, 9, 9, 10]
plana = list(range(1, 11))
picuda = [5, 5, 5, 5, 5, 5, 5, 5, 1, 9]
norm = np.random.default_rng(1).normal(size=5000)
```

FORMA	DESCRIPCIÓN	EJEMPLO	RESULTADO
Simétrica	skew cerca de 0: media igual a mediana.	<code>print(round(stats.skew(sim), 2))</code>	0.0
Asimetría positiva	skew > 0: cola a la derecha, media > mediana.	<code>print(round(stats.skew(der), 2))</code>	2.02
Asimetría negativa	skew < 0: cola a la izquierda, media < mediana.	<code>print(round(stats.skew(izq), 2))</code>	-1.5
Media y mediana	En la positiva la media supera a la mediana.	<code>print(round(np.mean(der), 1), np.median(der))</code>	892.9 550.0
Mesocúrtica	kurtosis ≈ 0: tan puntiaguda como la normal.	<code>print(round(stats.kurtosis(norm), 1))</code>	0.1
Platicúrtica	kurtosis < 0: más plana que la normal.	<code>print(round(stats.kurtosis(plana), 2))</code>	-1.22
Leptocúrtica	kurtosis > 0: más picuda, con colas pesadas.	<code>print(round(stats.kurtosis(picuda), 2))</code>	2.0

FORMA	DESCRIPCIÓN	EJEMPLO	RESULTADO
<code>fisher=False</code>	Con <code>fisher=False</code> la normal vale 3 (Pearson).	<code>print(round(stats.kurtosis(picuda, fisher=False), 2))</code>	5.0

scipy da la kurtosis en exceso: se le resta 3 para que la normal valga 0.

TEMA 10

Diagrama de caja y datos atípicos

El diagrama de caja resume los datos con 5 números: mínimo, Q1, mediana, Q3 y máximo. Un dato es atípico si cae fuera de $[Q1 - 1.5 \cdot IQR, Q3 + 1.5 \cdot IQR]$.

RECUERDA La caja es donde vive el 50 % central; los bigotes son los vecinos normales; los puntos sueltos son los raros.

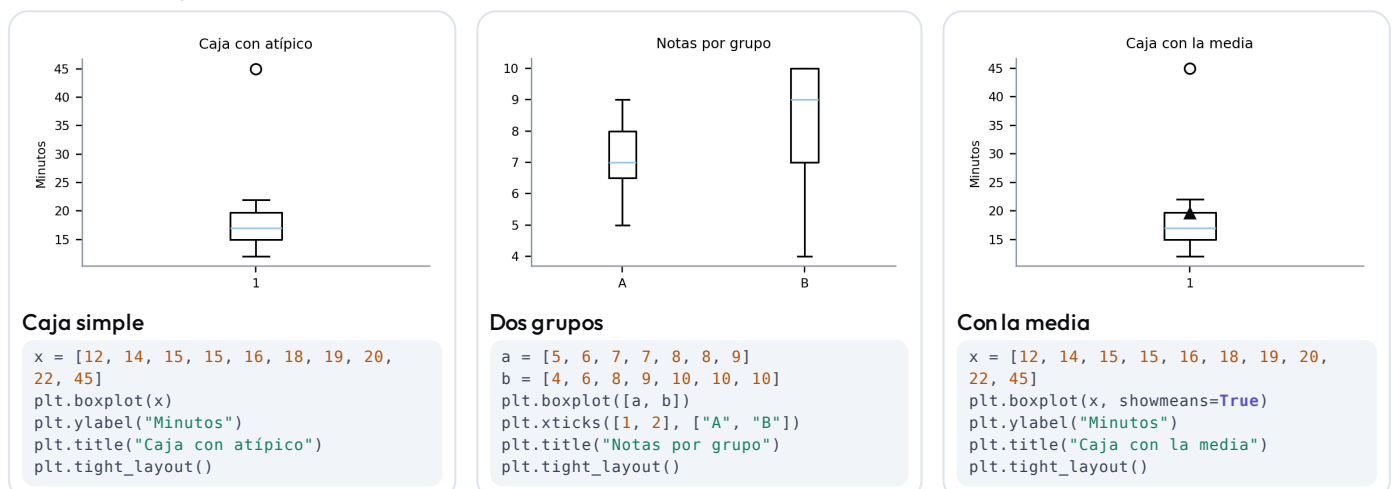
DATOS DE EJEMPLO

```
import numpy as np
import matplotlib.pyplot as plt
x = np.array([12, 14, 15, 15, 16, 18, 19, 20, 22, 45])
```

ELEMENTO	DESCRIPCIÓN	EJEMPLO	RESULTADO
Cinco números	Mínimo, Q1, mediana, Q3 y máximo.	<code>print(np.percentile(x, [0, 25, 50, 75, 100]))</code>	[12. 15. 17. 19.75 45.]
IQR	$IQR = Q3 - Q1$: el ancho de la caja.	<code>q1, q3 = np.percentile(x, [25, 75])</code> <code>print(q3 - q1)</code>	4.75
Valla inferior	$Q1 - 1.5 \cdot IQR$.	<code>q1, q3 = np.percentile(x, [25, 75])</code> <code>print(q1 - 1.5 * (q3 - q1))</code>	7.875
Valla superior	$Q3 + 1.5 \cdot IQR$.	<code>q1, q3 = np.percentile(x, [25, 75])</code> <code>print(q3 + 1.5 * (q3 - q1))</code>	26.875
Detectar atípicos	Datos fuera de las vallas.	<code>q1, q3 = np.percentile(x, [25, 75])</code> <code>i = q3 - q1</code> <code>bajo = x < q1 - 1.5 * i</code> <code>alto = x > q3 + 1.5 * i</code> <code>print(x[bajo alto])</code>	[45]
Bigote superior	Mayor dato que no pasa la valla superior.	<code>print(x[x <= 26.875].max())</code>	22
plt.boxplot()	Dibuja la caja, los bigotes y los atípicos.	<code>plt.boxplot(x)</code>	Caja de 15 a 19.75 y un punto en 45

Los datos son minutos de viaje en bus de 10 estudiantes.

Gráficos de ejemplo



Correlación entre dos variables

La correlación mide qué tan ligadas están dos variables numéricas. El coeficiente de Pearson r va de -1 a 1 : cerca de 1 sube junto, cerca de -1 una sube y la otra baja, cerca de 0 no hay relación lineal.

RECUERDA Más horas de estudio, mejor nota: r positivo. Más horas en el celular, peor nota: r negativo.

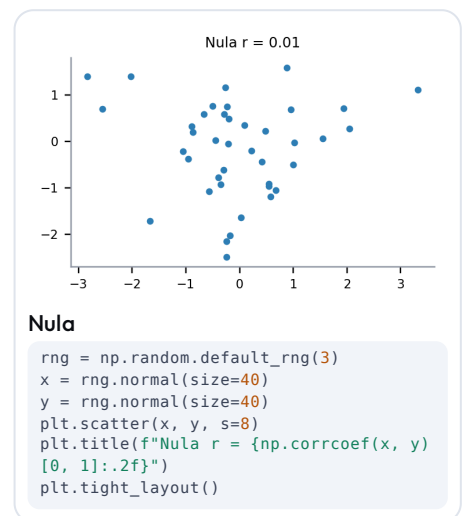
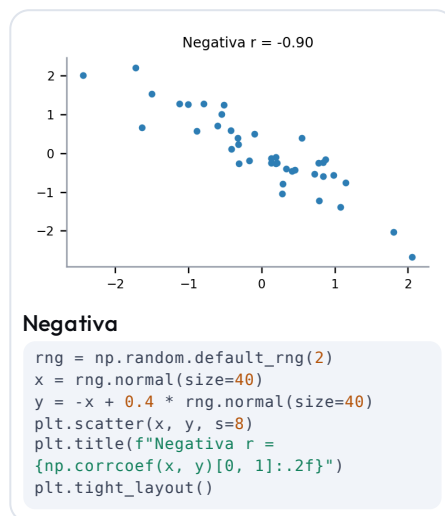
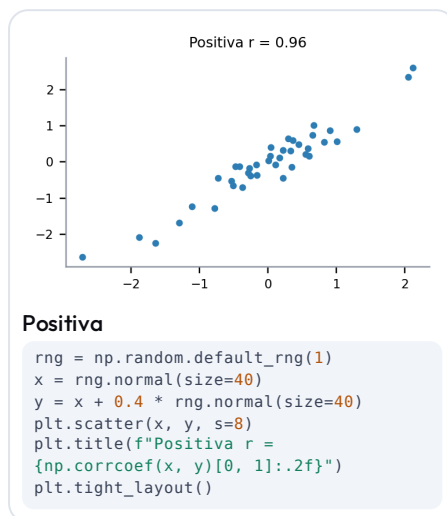
DATOS DE EJEMPLO

```
import numpy as np
from scipy import stats
h = np.array([1, 2, 3, 4, 5, 6])
n = np.array([4, 5, 5, 7, 8, 9])
```

MEDIDA	DESCRIPCIÓN	EJEMPLO	RESULTADO
Covarianza	$\text{cov} = \sum (x - \text{media } x)(y - \text{media } y) / (n - 1)$.	<code>print(round(np.cov(h, n)[0, 1], 3))</code>	3.6
Pearson (fórmula)	$r = \text{cov} / (\text{sx} \cdot \text{sy})$. No tiene unidades.	<code>c = np.cov(h, n)[0, 1]</code> <code>print(round(c / (h.std(ddof=1) * n.std(ddof=1)), 3))</code>	0.979
np.corrcoef()	Matriz de correlaciones; el 1 es de cada variable consigo misma.	<code>print(np.corrcoef(h, n).round(3))</code>	<code>[[1. 0.979]</code> <code>[0.979 1.]]</code>
stats.pearsonr()	Devuelve r (y también el valor p).	<code>print(round(float(stats.pearsonr(h, n)[0]), 3))</code>	0.979
r al cuadrado	r^2 = parte de la variación de y explicada por x .	<code>print(round(np.corrcoef(h, n)[0, 1] ** 2, 3))</code>	0.958
Relación inversa	Si una sube y la otra baja, r es negativo.	<code>print(round(np.corrcoef(h, n[::-1])[0, 1], 3))</code>	-0.979
Cómo leer r	Fuerza según el valor absoluto de r .	$ r $	0 a 0.3: débil 0.3 a 0.7: moderada 0.7 a 1: fuerte
Correlación no es causa	Que dos variables suban juntas no prueba que una cause la otra.	Helados y quemaduras de sol	Ambos suben en verano, sin causa

h son las horas de estudio y n la nota de 6 estudiantes.

Gráficos de ejemplo



Métodos de enumeración y probabilidad

Contar posibilidades y medir la incertidumbre con probabilidades.

TEMA 12

Experimento aleatorio, espacio muestral y eventos

Un experimento aleatorio tiene resultados que no podemos predecir con certeza, como lanzar un dado. El espacio muestral S es el conjunto de todos los resultados posibles y un evento es un subconjunto de S.

RECUERDA Espacio muestral = el menú completo. Evento = el grupo de platos que te interesa pedir.

DATOS DE EJEMPLO

```
from itertools import product
dado = {1, 2, 3, 4, 5, 6}
moneda = {"C", "S"}
```

CONCEPTO	DESCRIPCIÓN	EJEMPLO	RESULTADO
Espacio muestral S	Todos los resultados posibles del dado.	<code>print(dado, len(dado))</code>	{1, 2, 3, 4, 5, 6} 6
Dos monedas	S con los resultados de lanzar 2 monedas.	<code>S2 = product("CS", repeat=2)</code> <code>print(["".join(p) for p in S2])</code>	['CC', 'CS', 'SC', 'SS']
Dos dados	Cantidad de resultados al lanzar 2 dados.	<code>print(len(list(product(dado, repeat=2))))</code>	36
Evento simple	Tiene un solo resultado (sale 4).	<code>A = {4}</code> <code>print(A <= dado, len(A))</code>	True 1
Evento compuesto	Tiene varios resultados (sale par).	<code>B = {x for x in dado if x % 2 == 0}</code> <code>print(B)</code>	{2, 4, 6}
Evento seguro	Es todo S: siempre ocurre.	<code>E = {x for x in dado if x > 0}</code> <code>print(E == dado)</code>	True
Evento imposible	No tiene resultados: es el conjunto vacío.	<code>I = {x for x in dado if x > 6}</code> <code>print(I, I == set())</code>	set() True
Cuántos eventos hay	Si S tiene n resultados, hay 2^n eventos.	<code>print(2 ** len(dado))</code>	64

TEMA 13

Operaciones con eventos

Como los eventos son conjuntos, se combinan igual: unión (A o B), intersección (A y B), complemento (no A) y diferencia (A pero no B). Dos eventos son mutuamente excluyentes si no comparten resultados.

RECUERDA Unión = todos los que están en alguno. Intersección = los que están en ambos a la vez.

DATOS DE EJEMPLO

```
S = set(range(1, 11))
A = {2, 4, 6, 8, 10}
B = {1, 2, 3, 4, 5}
```

OPERACIÓN	DESCRIPCIÓN	EJEMPLO	RESULTADO
Unión A ∪ B	Ocorre A, o B, o ambos.	<code>print(A B)</code>	{1, 2, 3, 4, 5, 6, 8, 10}
Intersección A ∩ B	Ocurren A y B a la vez.	<code>print(A & B)</code>	{2, 4}
Complemento A'	Todo lo de S que no está en A.	<code>print(S - A)</code>	{1, 3, 5, 7, 9}

OPERACIÓN	DESCRIPCIÓN	EJEMPLO	RESULTADO
Diferencia A - B	Está en A pero no en B.	<code>print(A - B)</code>	{8, 10, 6}
Diferencia simétrica	En uno solo de los dos eventos.	<code>print(A ^ B)</code>	{1, 3, 5, 6, 8, 10}
Mutuamente excluyentes	No tienen nada en común: $A \cap B$ vacío.	<code>C = {1, 3, 7}</code> <code>print(A.isdisjoint(C), A.isdisjoint(B))</code>	True False
Leyes de De Morgan	$(A \cup B)' = A' \cap B'$.	<code>print(S - (A B) == (S - A) & (S - B))</code>	True
Conteo de la unión	$n(A \cup B) = n(A) + n(B) - n(A \cap B)$.	<code>print(len(A B), len(A) + len(B) - len(A & B))</code>	8 8

S son 10 estudiantes numerados. A: número par. B: número menor o igual a 5.

TEMA 14

Principios de conteo

Regla del producto: si una tarea tiene pasos seguidos con m, n, ... formas, el total es $m \cdot n \cdot \dots$ Regla de la suma: si debes elegir una opción u otra (sin repetir), se suman las formas.

RECUERDA Y = multiplicar (paso tras paso). O = sumar (una opción u otra).

DATOS DE EJEMPLO

```
from itertools import product
camisas = ["roja", "azul", "negra"]
pantalones = ["jean", "short"]
```

IDEA	DESCRIPCIÓN	EJEMPLO	RESULTADO
Regla del producto	Camisas y pantalones: $m \cdot n$ combinaciones.	<code>print(len(camisas) * len(pantalones))</code>	6
Diagrama de árbol	Lista cada rama (camisa, pantalón).	<code>for c, p in product(camisas, pantalones):</code> <code>print(c, p)</code>	roja jean roja short azul jean azul short negra jean negra short
Regla de la suma	Ir a ESPO en bus o en taxi: $m + n$.	<code>buses = ["L1", "L2", "L3"]</code> <code>taxis = ["T1", "T2"]</code> <code>print(len(buses) + len(taxis))</code>	5
Tres pasos	Menú: 3 entradas, 4 platos, 2 postres.	<code>print(3 * 4 * 2)</code>	24
Con repetición	Clave de 3 dígitos (0 a 9): $10 ** 3$.	<code>claves = product(range(10), repeat=3)</code> <code>print(len(list(claves)))</code>	1000
Sin repetición	Clave de 3 dígitos distintos: $10 \cdot 9 \cdot 8$.	<code>print(10 * 9 * 8)</code>	720
Tres monedas	Resultados al lanzar 3 monedas: $2 ** 3$.	<code>print(len(list(product("CS", repeat=3))))</code>	8
Cuál regla usar	Y → producto. O → suma.	Pasos seguidos → multiplica Opciones alternas → suma	Producto: pasos. Suma: opciones.

Factorial y permutaciones

Una permutación es un orden: importa quién va primero. $n! = n \cdot (n-1) \cdot \dots \cdot 1$ cuenta las formas de ordenar n objetos y $nPr = n! / (n-r)!$ las formas de ordenar r de n .

RECUERDA Permutar = hacer una fila: cambiar el orden de las personas es otra fila distinta.

DATOS DE EJEMPLO

```
import math
from itertools import permutations
```

CONCEPTO	DESCRIPCIÓN	EJEMPLO	RESULTADO
$n!$	Factorial: $5! = 5 \cdot 4 \cdot 3 \cdot 2 \cdot 1$.	<code>print(math.factorial(5))</code>	120
$0!$	Por definición vale 1.	<code>print(math.factorial(0))</code>	1
Ordenar n objetos	4 amigos en una fila: $4!$.	<code>print(math.factorial(4))</code>	24
nPr	$nPr = n! / (n-r)!$. Podio de 3 entre 8.	<code>print(math.perm(8, 3))</code>	336
Comprobar fórmula	$8! / 5!$ da lo mismo que nPr .	<code>a = math.factorial print(a(8) // a(5))</code>	336
Listar permutaciones	Todas las formas de ordenar A, B, C.	<code>for p in permutations("ABC"): print("".join(p))</code>	ABC ACB BAC BCA CAB CBA
Con repetidos	$n! / (n_1! \cdot n_2!)$. Letras de CASA.	<code>a = math.factorial print(a(4) // a(2), len(set(permutations("CASA"))))</code>	12 12
Circulares	n personas en mesa redonda: $(n-1)!$.	<code>print(math.factorial(5))</code>	120

Mesa redonda de 6 personas: $(6-1)! = 120$.

Combinaciones

Una combinación es un grupo donde el orden no importa. $nCr = n! / (r! \cdot (n-r)!)$ cuenta cuántos grupos de r se pueden formar con n elementos.

RECUERDA Combinar = armar un equipo: da igual quién se escoge primero, el equipo es el mismo.

DATOS DE EJEMPLO

```
import math
from itertools import combinations
```

CONCEPTO	DESCRIPCIÓN	EJEMPLO	RESULTADO
nCr	Comité de 3 entre 10 estudiantes.	<code>print(math.comb(10, 3))</code>	120
Fórmula	$nCr = nPr / r!$	<code>print(math.perm(10, 3) // math.factorial(3))</code>	120
Permutar vs combinar	Con 5 y $r = 2$, nPr cuenta el orden.	<code>print(math.perm(5, 2), math.comb(5, 2))</code>	20 10
Listar grupos	Grupos de 2 entre A, B y C.	<code>for c in combinations("ABC", 2): print(c)</code>	('A', 'B') ('A', 'C') ('B', 'C')
Simetría	$nCr = nC(n-r)$.	<code>print(math.comb(10, 3), math.comb(10, 7))</code>	120 120

CONCEPTO	DESCRIPCIÓN	EJEMPLO	RESULTADO
Triángulo de Pascal	$nCr = (n-1)C(r-1) + (n-1)Cr$.	<code>print(math.comb(6, 2), math.comb(5, 1) + math.comb(5, 2))</code>	15 15
Binomio $(a+b)^4$	Los coeficientes son $4Ck$.	<code>print([math.comb(4, k) for k in range(5)])</code>	[1, 4, 6, 4, 1]
Manos de póker	Manos de 5 cartas de una baraja de 52.	<code>print(math.comb(52, 5))</code>	2598960

La suma de todos los nCr da $2^{**}n$ (todos los subconjuntos).

TEMA 17

Definición de probabilidad

La probabilidad mide qué tan posible es un evento, de 0 a 1. En la definición clásica, $P(A)$ = casos favorables / casos posibles (si todos son igual de probables). La frecuentista usa la frecuencia relativa al repetir muchas veces.

RECUERDA 0 = imposible, 1 = seguro. Mientras más cerca de 1, más probable.

DATOS DE EJEMPLO

```
from fractions import Fraction
from itertools import product
import numpy as np
```

IDEA	DESCRIPCIÓN	EJEMPLO	RESULTADO
Clásica	Casos favorables / posibles: corazón en una baraja.	<code>print(Fraction(13, 52))</code>	1/4
Con conjuntos	Sale par en un dado: $ A / S $.	<code>S = set(range(1, 7)) A = {2, 4, 6} print(len(A) / len(S))</code>	0.5
Dos dados	Suma 7 con dos dados.	<code>d = range(1, 7) S = list(product(d, repeat=2)) A = [s for s in S if sum(s) == 7] print(Fraction(len(A), len(S)))</code>	1/6
Frecuentista	Frecuencia relativa de pares en 10 000 tiros.	<code>rng = np.random.default_rng(1) t = rng.integers(1, 7, 10000) print(round((t % 2 == 0).mean(), 4))</code>	0.492
Axiomas 1 y 2	$0 \leq P(A) \leq 1$ y $P(S) = 1$.	<code>p = Fraction(1, 6) print(0 <= p <= 1, 6 * p)</code>	True 1
Axioma 3	Excluyentes: $P(A \cup B) = P(A) + P(B)$.	<code>p1 = Fraction(1, 6) p23 = Fraction(2, 6) print(p1 + p23)</code>	1/2
Complemento	$P(A') = 1 - P(A)$. Al menos una cara en 3 monedas.	<code>print(1 - Fraction(1, 2) ** 3)</code>	7/8

Regla de la suma de probabilidades

Para la probabilidad de que ocurra A o B: $P(A \cup B) = P(A) + P(B) - P(A \cap B)$. Se resta la intersección porque se contó dos veces. Si A y B son excluyentes, esa resta vale 0.

RECUERDA Dos listas de invitados: si un amigo está en ambas, no lo cuentes dos veces.

DATOS DE EJEMPLO

```
from fractions import Fraction
S = set(range(1, 7))
A = {2, 4, 6}
B = {4, 5, 6}
P = lambda E: Fraction(len(E), len(S))
```

CÁLCULO	DESCRIPCIÓN	EJEMPLO	RESULTADO
$P(A)$	Dado: sale par {2, 4, 6}.	<code>print(P(A))</code>	1/2
$P(B)$	Dado: sale mayor que 3 {4, 5, 6}.	<code>print(P(B))</code>	1/2
$P(A \cap B)$	Sale par y mayor que 3: {4, 6}.	<code>print(P(A & B))</code>	1/3
Regla general	$P(A) + P(B) - P(A \cap B)$.	<code>print(P(A) + P(B) - P(A & B))</code>	2/3
Comprobación directa	Cuenta los resultados de $A \cup B$.	<code>print(P(A B))</code>	2/3
Olvidar la resta	Error común: sumar sin restar da otro valor.	<code>print(P(A) + P(B))</code>	1
Excluyentes	Si $A \cap C$ es vacío, solo se suma.	<code>C = {1, 3}</code> <code>print(P(A C), P(A) + P(C))</code>	5/6 5/6
Cartas	Corazón o figura: $13/52 + 12/52 - 3/52$.	<code>c, f, cf = 13, 12, 3</code> <code>print(Fraction(c + f - cf, 52))</code>	11/26

Probabilidad condicional y regla del producto

$P(A|B)$ es la probabilidad de A sabiendo que ya ocurrió B: $P(A|B) = P(A \cap B) / P(B)$. De ahí sale la regla del producto: $P(A \cap B) = P(B) \cdot P(A|B)$.

RECUERDA Saber que B ocurrió achica el mundo: ahora solo cuentan los casos de B.

DATOS DE EJEMPLO

```
from fractions import Fraction
S = set(range(1, 7))
A = {2, 4, 6}
B = {4, 5, 6}
P = lambda E: Fraction(len(E), len(S))
```

CÁLCULO	DESCRIPCIÓN	EJEMPLO	RESULTADO
$P(B)$	Dado: sale mayor que 3 {4, 5, 6}.	<code>print(P(B))</code>	1/2
$P(A \cap B)$	Par y mayor que 3: {4, 6}.	<code>print(P(A & B))</code>	1/3
$P(A B)$	$P(A \cap B) / P(B)$.	<code>print(P(A & B) / P(B))</code>	2/3
Espacio reducido	Cuenta solo dentro de B.	<code>print(round(len(A & B) / len(B), 4))</code>	0.6667
Regla del producto	$P(B) \cdot P(A B)$ recupera $P(A \cap B)$.	<code>print(P(B) * Fraction(2, 3))</code>	1/3
El orden importa	Con $C = \{6\}$: $P(C A)$ y $P(A C)$ son distintas.	<code>C = {6}</code> <code>q = P(C & A)</code> <code>print(q / P(A), q / P(C))</code>	1/3 1

CÁLCULO	DESCRIPCIÓN	EJEMPLO	RESULTADO
Con conteos	60 mujeres, 24 de ellas en ingeniería.	<code>print(Fraction(24, 60))</code>	2/5
Sin reemplazo	Dos ases seguidos de una baraja.	<code>a = Fraction(4, 52)</code> <code>b = Fraction(3, 51)</code> <code>print(a * b)</code>	1/221

TEMA 20

Independencia de eventos

A y B son independientes si saber que ocurrió uno no cambia la probabilidad del otro: $P(A \cap B) = P(A) \cdot P(B)$. Ojo: ser independientes NO es lo mismo que ser mutuamente excluyentes.

RECUERDA Independientes = no se afectan (moneda y dado). Excluyentes = no pueden pasar juntos.

DATOS DE EJEMPLO

```
from itertools import product
from fractions import Fraction
S = set(product("CS", range(1, 7)))
P = lambda E: Fraction(len(E), len(S))
A = {s for s in S if s[0] == "C"}
B = {s for s in S if s[1] % 2 == 0}
```

CÁLCULO	DESCRIPCIÓN	EJEMPLO	RESULTADO
P(A) y P(B)	A: sale cara. B: el dado sale par.	<code>print(P(A), P(B))</code>	1/2 1/2
P(A ∩ B)	Cara y dado par: 3 de 12 casos.	<code>print(P(A & B))</code>	1/4
Prueba de independencia	¿ $P(A \cap B) = P(A) \cdot P(B)$?	<code>print(P(A & B) == P(A) * P(B))</code>	True
P(A B) = P(A)	Saber B no cambia la probabilidad de A.	<code>print(P(A & B) / P(B), P(A))</code>	1/2 1/2
Varios eventos	Tres caras seguidas: se multiplica.	<code>print(Fraction(1, 2) ** 3)</code>	1/8
Excluyentes	A y C (sale sello) nunca juntos, no son independientes.	<code>C = {s for s in S if s[0] == "S"}</code> <code>print(P(A & C), P(A) * P(C))</code>	0 1/4
Dependientes	E = dado mayor que 3: $P(B \cap E) \neq P(B) \cdot P(E)$.	<code>E = {s for s in S if s[1] > 3}</code> <code>print(P(B & E), P(B) * P(E))</code>	1/3 1/4

S = moneda y dado juntos: 12 resultados.

TEMA 21

Probabilidad total

Si B1, B2, ... forman una partición de S (no se cruzan y juntan todo S), entonces $P(D) = P(B1) \cdot P(D|B1) + P(B2) \cdot P(D|B2) + \dots$ Es un promedio ponderado de cada camino.

RECUERDA Un pastel en trozos: sumas lo que aporta cada trozo al total.

DATOS DE EJEMPLO

```
import numpy as np
p = [0.5, 0.3, 0.2]
d = [0.02, 0.03, 0.05]
a = [x * y for x, y in zip(p, d)]
```

CÁLCULO	DESCRIPCIÓN	EJEMPLO	RESULTADO
Partición	Las fábricas producen el 100% entre todas.	<code>print(round(sum(p), 4))</code>	1.0
Aporte de cada una	$P(Bi) \cdot P(D Bi)$ por fábrica.	<code>a = [x * y for x, y in zip(p, d)]</code> <code>print([round(v, 4) for v in a])</code>	[0.01, 0.009, 0.01]

CÁLCULO	DESCRIPCIÓN	EJEMPLO	RESULTADO
Probabilidad total	Suma de los aportes: P(defectuoso).	<code>a = [x * y for x, y in zip(p, d)] print(round(sum(a), 4))</code>	0.029
En porcentaje	Se lee: 2.9% de las piezas salen defectuosas.	<code>print(f"{sum(a):.1%}")</code>	2.9%
Con producto punto	Lo mismo con NumPy.	<code>print(round(np.dot(p, d), 4))</code>	0.029
Error: sin pesos	Promediar las tasas sin ponderar es incorrecto.	<code>print(round(sum(d) / 3, 4))</code>	0.0333
Fórmula	Suma ponderada sobre la partición.	$P(D) = P(F1)P(D F1) + P(F2)P(D F2) + P(F3)P(D F3)$	Suma de cada camino hasta D

Fábricas F1, F2, F3 producen 50%, 30% y 20%. Defectuosas: 2%, 3% y 5%.

TEMA 22

Teorema de Bayes

Bayes invierte una condicional: $P(A|B) = P(B|A) \cdot P(A) / P(B)$, con $P(B)$ por probabilidad total. Pasa de la probabilidad a priori (antes de ver el dato) a la probabilidad a posteriori (después de verlo).

RECUERDA Una prueba positiva no es una sentencia: pesa más cuántos enfermos hay que lo buena que es la prueba.

DATOS DE EJEMPLO

prev = 0.01
sens = 0.95
fp = 0.05

PASO	DESCRIPCIÓN	EJEMPLO	RESULTADO
Fórmula de Bayes	Prior + evidencia = posterior.	$P(E +) = P(+ E) \cdot P(E) / P(+)$	Posterior = prior · evidencia
Positivo y enfermo	$P(E) \cdot P(+ E) = \text{prev} \cdot \text{sens}$.	<code>print(round(prev * sens, 4))</code>	0.0095
Positivo y sano	$P(\text{sano}) \cdot P(+ \text{sano}) = (1-\text{prev}) \cdot \text{fp}$.	<code>print(round((1 - prev) * fp, 4))</code>	0.0495
P(+) total	Probabilidad total de dar positivo.	<code>print(round(prev * sens + (1 - prev) * fp, 4))</code>	0.059
P(E +) posterior	Probabilidad de estar enfermo si dio positivo.	<code>num = prev * sens den = num + (1 - prev) * fp print(round(num / den, 4))</code>	0.161
10 000 personas	Contar casos: verdaderos y falsos positivos.	<code>n = 10000 ver = round(n * prev * sens) fal = round(n * (1 - prev) * fp) print(ver, fal, round(ver / (ver + fal), 4))</code>	95 495 0.161
Enfermedad común	Con prev = 20% el posterior sube mucho.	<code>num = 0.2 * sens den = num + 0.8 * fp print(f"{num / den:.1%}")</code>	82.6%
Filtro de spam	40% spam. «gratis» en 60% del spam y 5% del normal.	<code>s, g, h = 0.4, 0.6, 0.05 num = s * g den = num + (1 - s) * h print(round(num / den, 4))</code>	0.8889

Prueba médica: prevalencia 1%, sensibilidad 95%, falsos positivos 5%.

Variables aleatorias discretas

Modelos para contar: éxitos, fallas y eventos que ocurren.

TEMA 23

Variable aleatoria discreta y función de masa

Una variable aleatoria discreta X asigna un número a cada resultado de un experimento y solo toma valores contables (0, 1, 2...). Su función de masa $p(x) = P(X = x)$ dice qué tan probable es cada valor.

RECUERDA pmf = tabla de reparto: la probabilidad total (1) se reparte entre los valores posibles, como dividir una pizza.

DATOS DE EJEMPLO

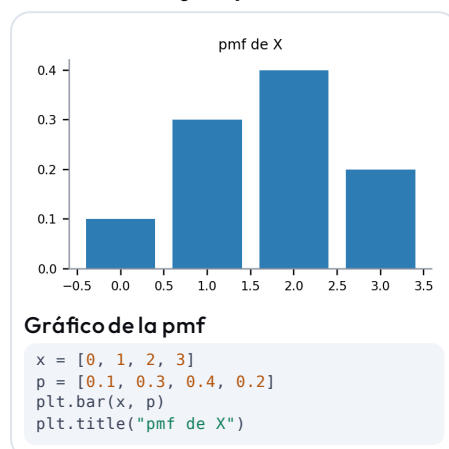
$x = [0, 1, 2, 3]$

$p = [0.1, 0.3, 0.4, 0.2]$

CONCEPTO	DESCRIPCIÓN	EJEMPLO	RESULTADO
Variable aleatoria X	Número que depende del azar. Ej: materias reprobadas.	X = materias reprobadas en un semestre	X toma los valores 0, 1, 2 o 3
$p(x) = P(X = x)$	Probabilidad de que X valga justo x .	<code>print(p[2])</code>	0.4
Propiedad $p(x) \geq 0$	Ninguna probabilidad puede ser negativa.	<code>print(all(q >= 0 for q in p))</code>	True
Propiedad $\sum p(x) = 1$	Todas las probabilidades suman 1.	<code>print(round(sum(p), 4))</code>	1.0
Tabla de la pmf	Cada valor junto a su probabilidad.	<pre>for a, b in zip(x, p): print(a, b)</pre>	0 0.1 1 0.3 2 0.4 3 0.2
$P(X \geq 2)$	Se suman las $p(x)$ de los valores pedidos.	<code>print(round(p[2] + p[3], 4))</code>	0.6
$P(X < 2)$	Menor que 2 es solo 0 y 1.	<code>print(round(p[0] + p[1], 4))</code>	0.4
Una pmf inválida	Si la suma no es 1, no es una pmf.	<code>print(round(sum([0.5, 0.3, 0.4]), 4))</code>	1.2

Aquí X = materias reprobadas por un estudiante; p da la probabilidad de cada valor.

Gráficos de ejemplo



Función de distribución acumulada

La acumulada $F(x) = P(X \leq x)$ suma las probabilidades hasta x . Siempre crece de 0 a 1 y avanza en escalones. Con ella se obtiene $P(a < X \leq b) = F(b) - F(a)$.

RECUERDA F es un contador de kilómetros: va sumando probabilidad y nunca retrocede.

DATOS DE EJEMPLO

```
import numpy as np
from scipy import stats
x = np.array([0, 1, 2, 3])
p = np.array([0.1, 0.3, 0.4, 0.2])
F = np.round(np.cumsum(p), 2)
```

CÁLCULO	DESCRIPCIÓN	EJEMPLO	RESULTADO
$F(x) = P(X \leq x)$	Suma acumulada de la pmf.	<code>print(np.round(np.cumsum(p), 2))</code>	[0.1 0.4 0.8 1.]
Leer $F(2)$	Probabilidad de 2 o menos.	<code>print(F[2])</code>	0.8
$P(X > 1)$	Complemento: $1 - F(1)$.	<code>print(round(1 - F[1], 4))</code>	0.6
$P(a < X \leq b)$	$F(b) - F(a)$. Aquí $P(0 < X \leq 2)$.	<code>print(round(F[2] - F[0], 4))</code>	0.7
$P(X = k)$	$F(k) - F(k-1)$. Aquí $P(X = 2)$.	<code>print(round(F[2] - F[1], 4))</code>	0.4
Entre valores	F es escalonada: $F(1.5) = F(1)$.	<code>print(F[x <= 1.5][-1])</code>	0.4
F en los extremos	F vale 0 antes del mínimo y 1 desde el máximo.	$F(-1) = 0$ y $F(5) = 1$	$F(-1) = 0$ y $F(5) = 1$
Con scipy	rv_discrete arma la distribución desde la tabla.	<code>d = stats.rv_discrete(values=(x, p))</code> <code>print(round(float(d.cdf(2)), 4))</code>	0.8

Esperanza o valor esperado

La esperanza $E[X] = \sum x \cdot p(x)$ es el promedio de X pesado por sus probabilidades. Si repites el experimento muchas veces, el promedio de los resultados se acerca a $E[X]$.

RECUERDA $E[X]$ es el punto de equilibrio de la pmf: donde se balancearía una regla con pesos $p(x)$.

DATOS DE EJEMPLO

```
import numpy as np
from scipy import stats
x = np.array([0, 1, 2, 3])
p = np.array([0.1, 0.3, 0.4, 0.2])
```

CÁLCULO	DESCRIPCIÓN	EJEMPLO	RESULTADO
$E[X] = \sum x \cdot p(x)$	Cada valor por su probabilidad, todo sumado.	<code>print(round(np.sum(x * p), 4))</code>	1.7
Con <code>np.dot()</code>	El mismo cálculo como producto punto.	<code>print(round(np.dot(x, p), 4))</code>	1.7
$E[X^2]$	$\sum x^2 \cdot p(x)$. Se usa para la varianza.	<code>print(round(np.sum(x**2 * p), 4))</code>	3.7
$E[g(X)]$	$\sum g(x) \cdot p(x)$. Ej: costo $g = 20x + 5$.	$g = 20 * x + 5$ <code>print(round(np.sum(g * p), 4))</code>	39.0
$E[g(X)] \neq g(E[X])$	Con g no lineal no se pueden intercambiar.	$a = np.sum(x**2 * p)$ $b = np.sum(x * p)**2$ <code>print(round(a, 4), round(b, 4))</code>	3.7 2.89

CÁLCULO	DESCRIPCIÓN	EJEMPLO	RESULTADO
Rifa de \$2	Premio neto 48 con prob 0.02; si no, pierdes 2.	<pre>g = np.array([48, -2]) q = np.array([0.02, 0.98]) print(round(np.sum(g * q), 4))</pre>	-1.0
Largo plazo	Con 10000 repeticiones, el promedio ronda $E[X]$.	<pre>rng = np.random.default_rng(1) s = rng.choice(x, 10000, p=p) print(round(float(s.mean()), 2))</pre>	1.71
Con scipy	Media de la distribución armada con la tabla.	<pre>d = stats.rv_discrete(values=(x, p)) print(round(float(d.mean()), 4))</pre>	1.7

TEMA 26

Varianza y desviación estándar

La varianza $\text{Var}(X) = E[X^2] - E[X]^2$ mide qué tan dispersos están los valores alrededor de la media. La desviación estándar σ es su raíz y viene en las mismas unidades de X .

RECUERDA Dos buses con el mismo tiempo promedio: el que a veces llega muy temprano y muy tarde tiene mayor varianza.

DATOS DE EJEMPLO

```
import numpy as np
from scipy import stats
x = np.array([0, 1, 2, 3])
p = np.array([0.1, 0.3, 0.4, 0.2])
m = np.sum(x * p)
v = np.sum(x**2 * p) - m**2
```

CÁLCULO	DESCRIPCIÓN	EJEMPLO	RESULTADO
$\text{Var} = E[X^2] - E[X]^2$	Fórmula corta de cálculo.	<pre>print(round(np.sum(x**2 * p) - m**2, 4))</pre>	0.81
$\text{Var} = \sum (x - \mu)^2 \cdot p(x)$	Definición: distancia al cuadrado a la media.	<pre>print(round(np.sum((x - m)**2 * p), 4))</pre>	0.81
$\sigma = \sqrt{\text{Var}}$	Desviación estándar.	<pre>print(round(float(np.sqrt(v)), 4))</pre>	0.9
$E[aX + b]$	$aE[X] + b$. Con $a = 20$, $b = 5$.	<pre>print(round(20 * m + 5, 4))</pre>	39.0
$\text{Var}(aX + b)$	$a^2 \text{Var}(X)$. Sumar b no cambia la dispersión.	<pre>print(round(20**2 * v, 4))</pre>	324.0
Comprobar $\text{Var}(aX + b)$	Cálculo directo con $g = 20x + 5$.	<pre>g = 20 * x + 5 mg = np.sum(g * p) print(round(np.sum((g - mg)**2 * p), 4))</pre>	324.0
$\text{Var}(X + 10)$	Correr los valores no cambia Var.	<pre>y = x + 10 my = np.sum(y * p) print(round(np.sum((y - my)**2 * p), 4))</pre>	0.81
Con scipy	Varianza y desviación de la tabla.	<pre>d = stats.rv_discrete(values=(x, p)) print(round(float(d.var()), 4)) print(round(float(d.std()), 4))</pre>	0.81 0.9

TEMA 27

Distribución uniforme discreta

Todos los valores posibles tienen la misma probabilidad $1/n$. Es el modelo de un dado justo, una ruleta o un sorteo al azar entre n personas.

RECUERDA Uniforme = reparto parejo: nadie tiene ventaja, como sacar un papelito de una urna bien mezclada.

DATOS DE EJEMPLO

```
import numpy as np
from scipy import stats
dado = np.arange(1, 7)
```

FUNCIÓN	DESCRIPCIÓN	EJEMPLO	RESULTADO
$p(x) = 1/n$	Dado de 6 caras: cada cara $1/6$.	<code>print(round(1 / 6, 4))</code>	0.1667
<code>stats.randint.pmf()</code>	<code>randint(a, b+1)</code> : el extremo superior NO se incluye.	<code>print(stats.randint.pmf(3, 1, 7).round(4))</code>	0.1667
<code>stats.randint.cdf()</code>	$P(X \leq 4)$ con un dado.	<code>print(stats.randint.cdf(4, 1, 7).round(4))</code>	0.6667
Media $(a + b)/2$	Valor central del dado.	<code>print(float(stats.randint.mean(1, 7)))</code>	3.5
Varianza $(n^2 - 1)/12$	Para $n = 6$: $35/12$.	<code>print(stats.randint.var(1, 7).round(4))</code>	2.9167
Cálculo directo	Media y varianza con las caras.	<code>print(dado.mean(), round(dado.var(), 4))</code>	3.5 2.9167
$P(\text{par})$	Casos favorables / casos totales = $3/6$.	<code>print(np.sum(dado % 2 == 0) / 6)</code>	0.5
Error: <code>randint(1, 6)</code>	Con <code>high = 6</code> la cara 6 nunca sale.	<code>print(stats.randint.pmf(6, 1, 6))</code>	0.0

TEMA 28

Distribución de Bernoulli

Es un solo ensayo con dos resultados: éxito ($X = 1$) con probabilidad p , o fracaso ($X = 0$) con probabilidad $1 - p$. Es el ladrillo con el que se construyen otros modelos.

RECUERDA Un bus llega a tiempo o no llega a tiempo: una sola pregunta de sí o no, con p = probabilidad del sí.

DATOS DE EJEMPLO

```
import numpy as np
from scipy import stats
p = 0.8
```

FUNCIÓN	DESCRIPCIÓN	EJEMPLO	RESULTADO
<code>pmf</code>	$P(X=0) = 1-p$ y $P(X=1) = p$.	<code>print(stats.bernoulli.pmf([0, 1], p))</code>	[0.2 0.8]
<code>cdf</code>	$P(X \leq 0) = 1-p$.	<code>print(stats.bernoulli.cdf(0, p).round(4))</code>	0.2
Media = p	$E[X] = p$.	<code>print(float(stats.bernoulli.mean(p)))</code>	0.8
Varianza = $p(1 - p)$	$0.8 \cdot 0.2$.	<code>print(stats.bernoulli.var(p).round(4))</code>	0.16
Desviación	$\sigma = \sqrt{p(1-p)}$.	<code>print(stats.bernoulli.std(p).round(4))</code>	0.4
Simular 10 ensayos	1 = llegó a tiempo. Con semilla fija.	<code>x = stats.bernoulli.rvs(p, size=10, random_state=5)</code> <code>print(x)</code>	[1 0 1 0 1 1 1 1 1 1]

FUNCIÓN	DESCRIPCIÓN	EJEMPLO	RESULTADO
Caso de la binomial	Bernoulli es binomial con $n = 1$.	<code>print(stats.binom.pmf(1, 1, p).round(4))</code>	0.8
Éxito no es «bueno»	Éxito es lo que se cuenta, aunque sea una falla.	$X = 1$ si el equipo falla	$p = P(\text{falla del equipo})$

Aquí $p = 0.8$ es la probabilidad de que el bus llegue a tiempo.

TEMA 29

Distribución binomial

Cuenta los éxitos en n ensayos independientes de Bernoulli, todos con la misma probabilidad p . $P(X = k) = C(n, k) \cdot p^k \cdot (1 - p)^{(n - k)}$. Media np y varianza npq , con $q = 1 - p$.

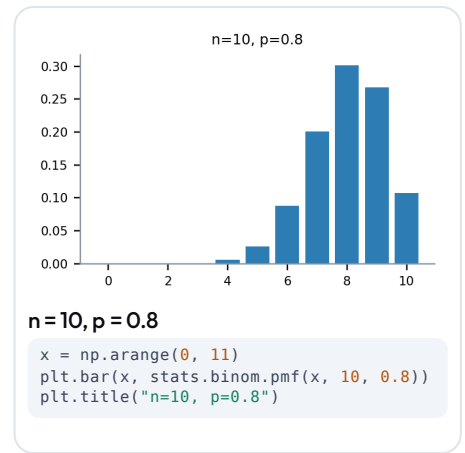
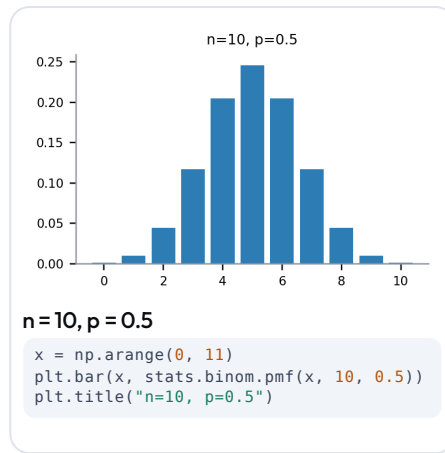
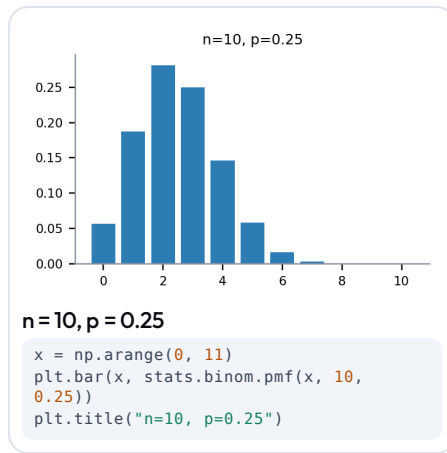
RECUERDA Examen de 10 preguntas con 4 opciones, contestando al azar: cuántas aciertas es binomial con $n = 10$ y $p = 0.25$.

DATOS DE EJEMPLO

```
import numpy as np
from scipy import stats
import math
n = 10
p = 0.25
```

FUNCIÓN	DESCRIPCIÓN	EJEMPLO	RESULTADO
stats.binom.pmf()	$P(X = 3)$: exactamente 3 aciertos. <code>pmf(k, n, p)</code> .	<code>print(stats.binom.pmf(3, n, p).round(4))</code>	0.2503
A mano	$C(10, 3) \cdot 0.25^3 \cdot 0.75^7$ con <code>math.comb</code> .	<code>c = math.comb(10, 3)</code> <code>print(round(c * 0.25**3 * 0.75**7, 4))</code>	0.2503
stats.binom.cdf()	$P(X \leq 3)$: 3 o menos aciertos.	<code>print(stats.binom.cdf(3, n, p).round(4))</code>	0.7759
$P(X \geq 6)$	Aprobar con 6 o más: $1 - F(5)$, o <code>sf(5)</code> .	<code>print(stats.binom.sf(5, n, p).round(4))</code>	0.0197
$P(2 \leq X \leq 4)$	$F(4) - F(1)$.	<code>a = stats.binom.cdf(4, n, p)</code> <code>b = stats.binom.cdf(1, n, p)</code> <code>print(round(float(a - b), 4))</code>	0.6778
Media y varianza	$np = 2.5$ y $npq = 1.875$.	<code>m, v = stats.binom.stats(n, p)</code> <code>print(float(m), float(v))</code>	2.5 1.875
Valor más probable	La moda es <code>floor((n+1)·p)</code> .	<code>k = np.arange(11)</code> <code>q = stats.binom.pmf(k, n, p)</code> <code>print(int(np.argmax(q)))</code>	2
Condiciones	n fijo, ensayos independientes, p constante.	$n = 10$, $p = 0.25$	Si fallan, no es binomial

Gráficos de ejemplo



TEMA 30

Distribución geométrica

Cuenta los ensayos necesarios hasta lograr el primer éxito. $P(X = k) = (1 - p)^{(k-1)} \cdot p$ con $k = 1, 2, 3, \dots$ Media $1/p$ y varianza $(1 - p)/p^2$.

RECUERDA Lanzas una moneda hasta que salga cara: cuántos lanzamientos necesitas es geométrico. Mientras menor p , más esperas.

DATOS DE EJEMPLO

```
import numpy as np
from scipy import stats
p = 0.1
```

FUNCIÓN	DESCRIPCIÓN	EJEMPLO	RESULTADO
<code>stats.geom.pmf()</code>	$P(X = 3)$: el primer defectuoso es el 3.º. $\text{pmf}(k, p)$.	<code>print(stats.geom.pmf(3, p).round(4))</code>	0.081
A mano	$0.9 \cdot 0.9 \cdot 0.1$: dos buenos y luego un defectuoso.	<code>print(round(0.9**2 * 0.1, 4))</code>	0.081
<code>stats.geom.cdf()</code>	$P(X \leq 5)$: aparece en los primeros 5.	<code>print(stats.geom.cdf(5, p).round(4))</code>	0.4095
$P(X > 5)$	Los primeros 5 salen buenos: $0.9^5 = \text{sf}(5)$.	<code>print(stats.geom.sf(5, p).round(4))</code>	0.5905
Media $1/p$	Se esperan 10 chips hasta el primer defectuoso.	<code>print(stats.geom.mean(p).round(4))</code>	10.0
Varianza $(1 - p)/p^2$	$0.9 / 0.01$.	<code>print(stats.geom.var(p).round(4))</code>	90.0
Sin memoria	$P(X > 8 X > 3) = P(X > 5)$.	<code>a = stats.geom.sf(8, p)</code> <code>b = stats.geom.sf(3, p)</code> <code>print(round(float(a / b), 4))</code>	0.5905
Empieza en 1	scipy no admite $k = 0$: la probabilidad es 0.	<code>print(stats.geom.pmf(0, p))</code>	0.0

Aquí $p = 0.1$ es la probabilidad de que un chip inspeccionado sea defectuoso.

TEMA 31

Distribución binomial negativa

Cuenta los ensayos hasta lograr el r -ésimo éxito. $P(T = t) = C(t-1, r-1) \cdot p^r \cdot (1-p)^{(t-r)}$ con $t = r, r+1, \dots$. Media r/p y varianza $r(1-p)/p^2$.

RECUERDA Un jugador quiere encestar 3 veces: cuántos tiros hace en total es binomial negativa con $r = 3$.

DATOS DE EJEMPLO

```
import numpy as np
from scipy import stats
import math
r = 3
p = 0.4
```

FUNCIÓN	DESCRIPCIÓN	EJEMPLO	RESULTADO
A mano	$P(T = 7)$: 3 encestes, el último en el tiro 7.	<code>c = math.comb(6, 2) print(round(c * 0.4**3 * 0.6**4, 4))</code>	0.1244
stats.nbinom.pmf()	scipy cuenta FALLAS: fallas = ensayos - $r = 7 - 3$.	<code>print(stats.nbinom.pmf(7 - r, r, p).round(4))</code>	0.1244
$P(T \leq 6)$	cdf con fallas = $6 - 3 = 3$.	<code>print(stats.nbinom.cdf(6 - r, r, p).round(4))</code>	0.4557
$P(T > 8)$	sf con fallas = $8 - 3 = 5$.	<code>print(stats.nbinom.sf(8 - r, r, p).round(4))</code>	0.3154
Media de ensayos	$r/p = 7.5$. scipy da 4.5 fallas; se suma r .	<code>f = stats.nbinom.mean(r, p) print(round(float(f) + r, 4))</code>	7.5
Varianza	$r(1-p)/p^2$: igual para fallas y ensayos.	<code>print(stats.nbinom.var(r, p).round(4))</code>	11.25
Error común	$\text{pmf}(7, r, p)$ es $P(7 \text{ fallas})$, o sea 10 ensayos.	<code>print(stats.nbinom.pmf(7, r, p).round(4))</code>	0.0645
Caso $r = 1$	Es la geométrica: $T = 3$ equivale a 2 fallas.	<code>a = stats.nbinom.pmf(2, 1, p) b = stats.geom.pmf(3, p) print(round(float(a), 4), round(float(b), 4))</code>	0.144 0.144

Aquí $p = 0.4$ es la probabilidad de encestar y $r = 3$ los encestes buscados.

TEMA 32

Distribución hipergeométrica

Cuenta los éxitos en una muestra de tamaño N tomada SIN reposición de una población de M elementos con n éxitos. $P(X = k) = \frac{C(n, k) \cdot C(M-n, N-k)}{C(M, N)}$. Media $N \cdot n/M$.

RECUERDA Sacar 4 chips de un lote sin devolverlos: cada extracción cambia el lote, por eso los ensayos ya no son independientes.

DATOS DE EJEMPLO

```
import numpy as np
from scipy import stats
import math
M = 20
n = 5
N = 4
```

FUNCIÓN	DESCRIPCIÓN	EJEMPLO	RESULTADO
A mano	$P(X = 2) = \frac{C(5, 2) \cdot C(15, 2)}{C(20, 4)}$.	<code>a = math.comb(5, 2) * math.comb(15, 2) print(round(a / math.comb(20, 4), 4))</code>	0.2167

FUNCIÓN	DESCRIPCIÓN	EJEMPLO	RESULTADO
<code>stats.hypergeom.pmf()</code>	Orden: pmf(k, M, n, N). Ver la última fila.	<code>print(stats.hypergeom.pmf(2, M, n, N).round(4))</code>	0.2167
$P(X \geq 1)$	Al menos un defectuoso: $1 - P(X = 0) = sf(0)$.	<code>print(stats.hypergeom.sf(0, M, n, N).round(4))</code>	0.7183
Media $N \cdot n / M$	$4 \cdot 5 / 20$.	<code>print(float(stats.hypergeom.mean(M, n, N)))</code>	1.0
Varianza	$N \cdot (n/M) \cdot (1 - n/M) \cdot (M - N) / (M - 1) = 12/19$.	<code>print(stats.hypergeom.var(M, n, N).round(4))</code>	0.6316
Comparar con binomial	Binomial (con reposición) da un valor cercano.	<code>print(stats.binom.pmf(2, N, n / M).round(4))</code>	0.2109
Población enorme	$M = 2000, n = 500$: casi igual a la binomial.	<code>a = stats.hypergeom.pmf(2, 2000, 500, N)</code> <code>print(round(float(a), 4))</code>	0.211
Orden de scipy	$M = \text{total}, n = \text{éxitos en el total}, N = \text{muestra}$.	<code>hypergeom(M, n, N)</code>	$M = 20, n = 5, N = 4$

Lote de $M = 20$ chips, $n = 5$ defectuosos; se revisan $N = 4$ sin devolverlos.

TEMA 33

Distribución de Poisson

Cuenta cuántas veces ocurre un evento en un intervalo de tiempo o espacio, con λ ocurrencias promedio. $P(X = k) = e^{(-\lambda)} \cdot \lambda^k / k!$. Media y varianza valen lo mismo: λ .

RECUERDA Llamadas por minuto a una central: llegan al azar, pero en promedio λ cada minuto. Media y varianza iguales es su huella.

```

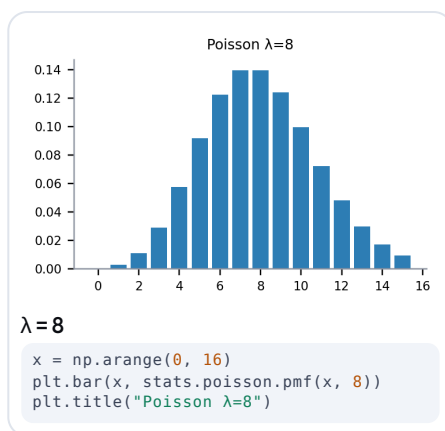
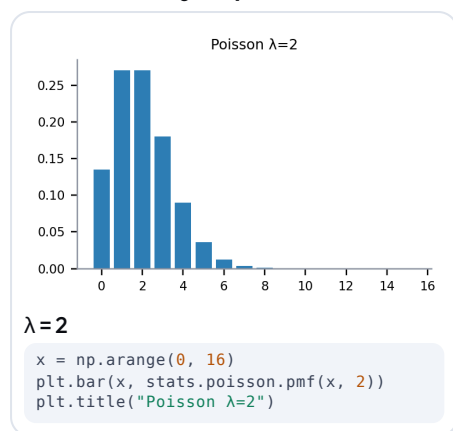
DATOS DE EJEMPLO
import numpy as np
from scipy import stats
import math
lam = 3

```

FUNCIÓN	DESCRIPCIÓN	EJEMPLO	RESULTADO
<code>stats.poisson.pmf()</code>	$P(X = 2)$ llamadas en un minuto. <code>pmf(k, λ)</code> .	<code>print(stats.poisson.pmf(2, lam).round(4))</code>	0.224
A mano	$e^{(-3)} \cdot 3^2 / 2!$.	<code>e = math.exp(-3)</code> <code>print(round(e * 3**2 / math.factorial(2), 4))</code>	0.224
<code>stats.poisson.cdf()</code>	$P(X \leq 2)$: 2 llamadas o menos.	<code>print(stats.poisson.cdf(2, lam).round(4))</code>	0.4232
$P(X \geq 5)$	5 o más: $1 - F(4) = sf(4)$.	<code>print(stats.poisson.sf(4, lam).round(4))</code>	0.1847
Media y varianza	Las dos son λ .	<code>m, v = stats.poisson.stats(lam)</code> <code>print(float(m), float(v))</code>	3.0 3.0
Cambiar el intervalo	En 2 minutos $\lambda = 6$. $P(0)$ llamadas).	<code>print(stats.poisson.pmf(0, 2 * lam).round(4))</code>	0.0025
Valor más probable	Con $\lambda = 3$ empatan 2 y 3.	<code>q = stats.poisson.pmf([2, 3], lam)</code> <code>print(np.round(q, 4))</code>	[0.224 0.224]
Condiciones	Eventos independientes, tasa constante, uno a la vez.	$\lambda = \text{tasa por intervalo}$	Ajusta λ al tamaño del intervalo

Aquí $\lambda = 3$ llamadas por minuto.

Gráficos de ejemplo



TEMA 34

Aproximación de Poisson a la binomial

Si n es grande y p pequeña, la binomial (n, p) se parece a una Poisson con $\lambda = np$. Sirve para eventos raros: defectos en un lote grande, fallas en muchas horas de uso.

RECUERDA Mucha gente intentando algo con muy poca probabilidad: como contar cuántos de 200 chips salen fallados con $p = 0.01$.

DATOS DE EJEMPLO

```

import numpy as np
from scipy import stats
n = 200
p = 0.01

```

CÁLCULO	DESCRIPCIÓN	EJEMPLO	RESULTADO
$\lambda = np$	El parámetro de la Poisson sale del producto.	<code>print(n * p)</code>	2.0
Binomial exacta	$P(X = 3)$ con $n = 200$, $p = 0.01$.	<code>print(stats.binom.pmf(3, n, p).round(4))</code>	0.1814
Poisson aproximada	$P(X = 3)$ con $\lambda = 2$.	<code>print(stats.poisson.pmf(3, n * p).round(4))</code>	0.1804
Diferencia	Error absoluto entre las dos.	<code>a = stats.binom.pmf(3, n, p)</code> <code>b = stats.poisson.pmf(3, n * p)</code> <code>print(round(float(abs(a - b)), 4))</code>	0.0009
$P(X \leq 2)$ binomial	Exacta.	<code>print(stats.binom.cdf(2, n, p).round(4))</code>	0.6767
$P(X \leq 2)$ Poisson	Aproximada: casi igual.	<code>print(stats.poisson.cdf(2, n * p).round(4))</code>	0.6767
Mayor error en toda la pmf	Máxima diferencia para $k = 0$ a 10.	<code>k = np.arange(0, 11)</code> <code>a = stats.binom.pmf(k, n, p)</code> <code>b = stats.poisson.pmf(k, n * p)</code> <code>print(round(float(np.max(abs(a - b))), 4))</code>	0.0014
Malá aproximación	$n = 10$, $p = 0.5$ (pnoes pequeña): $P(X = 3)$.	<code>a = stats.binom.pmf(3, 10, 0.5)</code> <code>b = stats.poisson.pmf(3, 5)</code> <code>print(round(float(a), 4), round(float(b), 4))</code>	0.1172 0.1404

Regla práctica: $n \geq 20$ y $p \leq 0.05$ (o np pequeño).

Cómo elegir el modelo

Para elegir, pregúntate qué se cuenta y cómo se hacen las observaciones: ¿un intento o varios?, ¿n fijo o se repite hasta un éxito?, ¿hay reposición?, ¿o se cuentan eventos en un intervalo?

RECUERDA Primero identifica qué cuentas (éxitos, intentos o eventos). Luego revisa si n es fijo y si hay reposición.

DATOS DE EJEMPLO

```
from scipy import stats
```

SITUACIÓN	DESCRIPCIÓN	EJEMPLO	RESULTADO
Todos igual de probables	Dado, ruleta, sorteo entre n personas.	<code>stats.randint(a, b + 1)</code>	Uniforme discreta
Un solo ensayo sí/no	Un bus llega o no llega a tiempo.	<code>stats.bernoulli(p)</code>	Bernoulli
Éxitos en n ensayos fijos	Aciertos en un examen de opción múltiple.	<code>stats.binom(n, p)</code>	Binomial
Ensayos hasta el primer éxito	Chips revisados hasta hallar un defectuoso.	<code>stats.geom(p)</code>	Geométrica
Ensayos hasta el r-ésimo éxito	Tiros hasta encestar 3 veces.	<code>stats.nbinom(r, p)</code>	Binomial negativa (scipy cuenta fallas)
Muestra sin reposición	Defectuosos en 4 chips tomados de un lote de 20.	<code>stats.hypergeom(M, n, N)</code>	Hipergeométrica
Eventos por intervalo	Llamadas por minuto, fallas por semana.	<code>stats.poisson(lam)</code>	Poisson
n grande y p pequeña	Defectos en 200 piezas con $p = 0.01$.	<code>stats.poisson(n * p)</code>	Poisson con $\lambda = np$

Variables aleatorias continuas

Modelos para medir: tiempos, alturas, pesos y otras cantidades continuas.

TEMA 36

Variable aleatoria continua y función de densidad

Una variable continua puede tomar cualquier valor de un intervalo (un tiempo, una altura). Su función de densidad $f(x)$ cumple $f(x) \geq 0$ y área total 1. Las probabilidades son áreas bajo f .

RECUERDA La densidad es como un mapa de lluvia: donde hay más «agua» hay más probabilidad, y la probabilidad real es el área, no la altura.

DATOS DE EJEMPLO

```
import numpy as np
from scipy import stats
from scipy.integrate import quad
f = lambda x: x / 2
```

IDEA	DESCRIPCIÓN	EJEMPLO	RESULTADO
$f(x) \geq 0$	La densidad nunca es negativa. Aquí: $f(x) = x/2$ en $[0, 2]$.	<code>print(f(0), f(1), f(2))</code>	0.0 0.5 1.0
Área total = 1	Integral de f sobre todo el rango = 1.	<code>round(quad(f, 0, 2)[0], 4)</code>	1.0
$P(X = x) = 0$	Un valor exacto tiene área cero.	<code>round(quad(f, 1, 1)[0], 4)</code>	0.0
$P(a < X < b)$	Área entre a y b : integral de f de a hasta b .	<code>round(quad(f, 0.5, 1.5)[0], 4)</code>	0.5
$P(X < b)$	Área desde el inicio hasta b .	<code>round(quad(f, 0, 1)[0], 4)</code>	0.25
$P(X > a)$	Área desde a hasta el final.	<code>round(quad(f, 1.5, 2)[0], 4)</code>	0.4375
$f(x)$ puede pasar de 1	La densidad no es probabilidad; el área sí ≤ 1 .	<code>g = lambda x: 2</code> <code>print(g(0.1), round(quad(g, 0, 0.5)[0], 4))</code>	2 1.0
Densidad inválida	Tiene área 1, pero $f(x) < 0$ en un tramo.	<code>h = lambda x: x - 0.5</code> <code>print(h(0), round(quad(h, 0, 2)[0], 4))</code>	-0.5 1.0

Ejemplo: X = minutos de espera de un bus, con $f(x) = x/2$ entre 0 y 2.

TEMA 37

Función de distribución acumulada continua

$F(x) = P(X \leq x)$ es el área de la densidad desde el inicio hasta x . Siempre sube de 0 a 1. La derivada de F devuelve la densidad f .

RECUERDA F es un «contador de lluvia»: va sumando el agua acumulada hasta el punto x . Nunca baja.

DATOS DE EJEMPLO

```
import numpy as np
from scipy import stats
from scipy.integrate import quad
f = lambda x: x / 2
F = lambda x: quad(f, 0, x)[0]
```

IDEA	DESCRIPCIÓN	EJEMPLO	RESULTADO
$F(x)$ = integral de f	Área hasta x . Con $f(x) = x/2$ resulta $x^2/4$.	<code>round(F(1), 4)</code>	0.25

IDEA	DESCRIPCIÓN	EJEMPLO	RESULTADO
F al final = 1	Al llegar al máximo se acumula todo.	<code>round(F(2), 4)</code>	1.0
F al inicio = 0	Antes del primer valor no hay área.	<code>round(F(0), 4)</code>	0.0
P(a < X < b)	$F(b) - F(a)$.	<code>round(F(1.5) - F(0.5), 4)</code>	0.5
P(X > a)	$1 - F(a)$.	<code>round(1 - F(1.5), 4)</code>	0.4375
Derivada de F = f	Pendiente de F en $x = 1$ (debe dar $f(1) = 0.5$).	<code>e = 1e-4</code> <code>round((F(1 + e) - F(1 - e)) / (2 * e), 4)</code>	0.5
Comprobar con $x^2/4$	La integral coincide con la fórmula.	<code>print(round(F(1.2), 4), 1.2 ** 2 / 4)</code>	0.36 0.36
Mediana: F(m) = 0.5	Con $m = \sqrt{2}$ la mitad del área queda a la izquierda.	<code>round(F(2 ** 0.5), 4)</code>	0.5

Mismo ejemplo del tema anterior: $f(x) = x/2$ en $[0, 2]$.

TEMA 38

Esperanza y varianza continuas

$E[X] = \text{integral de } x \cdot f(x) \, dx$ es el valor promedio esperado. $\text{Var}(X) = E[X^2] - (E[X])^2$ mide la dispersión. Es lo mismo que en discretas, cambiando la suma Σ por una integral.

RECUERDA La esperanza es el punto de equilibrio de la densidad, como el centro de una balanza.

DATOS DE EJEMPLO

```
import numpy as np
from scipy import stats
from scipy.integrate import quad
f = lambda x: x / 2
```

MEDIDA	DESCRIPCIÓN	EJEMPLO	RESULTADO
E[X]	Integral de $x \cdot f(x)$. Con $f(x) = x/2$ en $[0, 2]$.	<code>round(quad(lambda x: x * f(x), 0, 2)[0], 4)</code>	1.3333
E[X²]	Integral de $x^2 \cdot f(x)$.	<code>round(quad(lambda x: x**2 * f(x), 0, 2)[0], 4)</code>	2.0
Var(X)	$E[X^2] - (E[X])^2 = 2 - (4/3)^2$.	<code>m = quad(lambda x: x * f(x), 0, 2)[0]</code> <code>round(2 - m ** 2, 4)</code>	0.2222
Desviación σ	Raíz cuadrada de la varianza.	<code>round((2 - (4 / 3) ** 2) ** 0.5, 4)</code>	0.4714
Var por definición	Integral de $(x - \mu)^2 \cdot f(x)$.	<code>m = 4 / 3</code> <code>round(quad(lambda x: (x - m)**2 * f(x), 0, 2)[0], 4)</code>	0.2222
E[aX + b]	$aE[X] + b$. Con $a = 2$ y $b = 3$.	<code>round(quad(lambda x: (2*x + 3) * f(x), 0, 2)[0], 4)</code>	5.6667
Var(aX + b)	$a^2 \cdot \text{Var}(X)$; sumar b no cambia la dispersión.	<code>mu = 2 * 4 / 3 + 3</code> <code>round(quad(lambda x: (2*x + 3 - mu)**2 * f(x), 0, 2)[0], 4)</code>	0.8889

Percentiles y cuantiles

El percentil p es el valor que deja a su izquierda una proporción p del área. La mediana es el percentil 50 y los cuartiles son los percentiles 25, 50 y 75. En `scipy` se calcula con `ppf`.

RECUERDA `ppf` es el «cdf al revés»: `cdf` pregunta «¿qué área hay hasta x ?» y `ppf` pregunta «¿hasta dónde llego para juntar esta área?».

DATOS DE EJEMPLO

```
import numpy as np
from scipy import stats
from scipy.integrate import quad
d = stats.norm(70, 10)
```

MEDIDA	DESCRIPCIÓN	EJEMPLO	RESULTADO
Mediana (Q2)	Deja 50% del área a cada lado. Notas con $\mu = 70$, $\sigma = 10$.	<code>round(float(d.ppf(0.5)), 4)</code>	70.0
Primer cuartil Q1	Deja 25% del área a la izquierda.	<code>round(float(d.ppf(0.25)), 4)</code>	63.2551
Tercer cuartil Q3	Deja 75% del área a la izquierda.	<code>round(float(d.ppf(0.75)), 4)</code>	76.7449
Rango intercuartílico	$IQR = Q3 - Q1$.	<code>round(float(d.ppf(0.75) - d.ppf(0.25)), 4)</code>	13.4898
Percentil 90	Solo 10% de las notas lo supera.	<code>round(float(d.ppf(0.90)), 4)</code>	82.8155
Percentil 95	Solo 5% de las notas lo supera.	<code>round(float(d.ppf(0.95)), 4)</code>	86.4485
ppf y cdf son inversas	<code>cdf(ppf(p))</code> devuelve p .	<code>x = d.ppf(0.9)</code> <code>round(float(d.cdf(x)), 4)</code>	0.9
Mediana exponencial	Tiempo de atención con media 5 min: $5 \cdot \ln 2$.	<code>round(float(stats.expon.ppf(0.5, scale=5)), 4)</code>	3.4657

Distribución uniforme continua

Todos los valores entre a y b son igual de probables. La densidad es constante: $f(x) = 1/(b - a)$. Media = $(a + b)/2$ y varianza = $(b - a)^2/12$.

RECUERDA Un reloj que se detiene en cualquier instante: todos los momentos tienen la misma «oportunidad».

DATOS DE EJEMPLO

```
import numpy as np
from scipy import stats
from scipy.integrate import quad
d = stats.uniform(loc=5, scale=10)
```

MEDIDA	DESCRIPCIÓN	EJEMPLO	RESULTADO
loc y scale	<code>loc = a</code> (inicio) y <code>scale = b - a</code> (ancho). Aquí $a = 5$, $b = 15$.	<code>a, b = d.support()</code> <code>print(float(a), float(b))</code>	5.0 15.0
Densidad	$f(x) = 1/(b - a) = 1/10$ dentro del rango.	<code>round(float(d.pdf(8)), 4)</code>	0.1
P(X < 8)	$(8 - 5)/(15 - 5) = 3/10$.	<code>round(float(d.cdf(8)), 4)</code>	0.3
P(7 < X < 12)	<code>cdf(12) - cdf(7) = 5/10</code> .	<code>round(float(d.cdf(12) - d.cdf(7)), 4)</code>	0.5
Media	$(a + b)/2 = 10$.	<code>round(float(d.mean()), 4)</code>	10.0
Varianza	$(b - a)^2/12 = 100/12$.	<code>round(float(d.var()), 4)</code>	8.3333
Fuera del rango	La densidad vale 0 fuera de $[a, b]$.	<code>round(float(d.pdf(20)), 4)</code>	0.0

MEDIDA	DESCRIPCIÓN	EJEMPLO	RESULTADO
Percentil 90	$a + 0.9 \cdot (b - a)$.	<code>round(float(d.ppf(0.9)), 4)</code>	14.0

Ejemplo: minutos de llegada de un profesor entre las 5 y las 15 después de la hora.

TEMA 41

Distribución exponencial

Modela el tiempo entre eventos (llegadas, fallas) que ocurren al azar con tasa λ por unidad de tiempo. Media = $1/\lambda$ y varianza = $1/\lambda^2$. En scipy se usa `scale = 1/\lambda`.

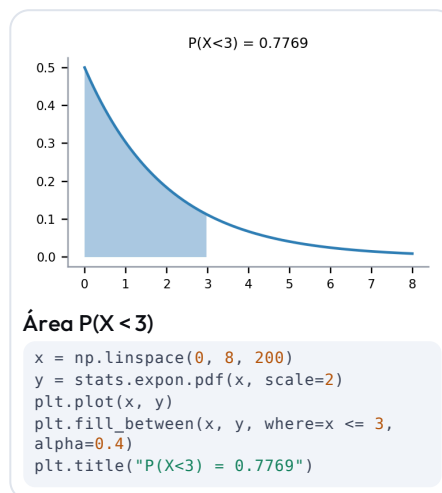
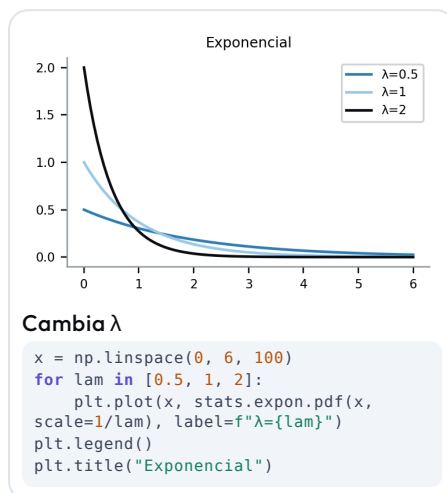
RECUERDA Si llegan $\lambda = 0.5$ clientes por minuto, entre uno y otro pasan en promedio $1/0.5 = 2$ minutos. En scipy escribe `scale`, no λ .

DATOS DE EJEMPLO

```
import numpy as np
from scipy import stats
from scipy.integrate import quad
lam = 0.5
d = stats.expon(scale=1 / lam)
```

MEDIDA	DESCRIPCIÓN	EJEMPLO	RESULTADO
Media	$1/\lambda$ minutos entre llegadas ($\lambda = 0.5$).	<code>round(float(d.mean()), 4)</code>	2.0
Varianza	$1/\lambda^2$.	<code>round(float(d.var()), 4)</code>	4.0
Densidad	$f(x) = \lambda \cdot \exp(-\lambda x)$. En $x = 1$.	<code>round(float(d.pdf(1)), 4)</code>	0.3033
$P(X < 3)$	$1 - \exp(-\lambda x) = 1 - \exp(-1.5)$.	<code>round(float(d.cdf(3)), 4)</code>	0.7769
$P(X > 3)$	$\exp(-\lambda x)$: la cola (<code>sf = 1 - cdf</code>).	<code>round(float(d.sf(3)), 4)</code>	0.2231
Falta de memoria	$P(X > 5 \mid X > 2) = P(X > 3)$.	<code>print(round(float(d.sf(5) / d.sf(2)), 4), round(float(d.sf(3)), 4))</code>	0.2231 0.2231
Mediana	$\ln 2 / \lambda$: la mitad llega antes.	<code>round(float(d.ppf(0.5)), 4)</code>	1.3863
Error: <code>scale = λ</code>	Con <code>scale = 0.5</code> la media sería 0.5 min.	<code>round(float(stats.expon(scale=0.5).mean()), 4)</code>	0.5

Gráficos de ejemplo



Distribución gamma

Modela el tiempo hasta que ocurre el evento número α (suma de α exponenciales). Tiene forma α y tasa β ; $\text{scale} = 1/\beta$. Media = α/β y varianza = α/β^2 . Si α es entero se llama Erlang.

RECUERDA Esperar a 3 clientes: cada espera es exponencial y la gamma suma las 3. Con $\alpha = 1$ vuelves a la exponencial.

DATOS DE EJEMPLO

```
import numpy as np
from scipy import stats
from scipy.integrate import quad
alfa, beta = 3, 0.5
d = stats.gamma(a=alfa, scale=1 / beta)
```

MEDIDA	DESCRIPCIÓN	EJEMPLO	RESULTADO
scale = $1/\beta$	Con $\alpha = 3$ y $\beta = 0.5$ clientes por minuto, $\text{scale} = 2$.	<code>print(d.kwds["scale"])</code>	2.0
Media	$\alpha/\beta = 3/0.5$.	<code>round(float(d.mean()), 4)</code>	6.0
Varianza	$\alpha/\beta^2 = 3/0.25$.	<code>round(float(d.var()), 4)</code>	12.0
Suma de exponenciales	Simula 3 esperas (media 2) y las suma.	<code>g = np.random.default_rng(1)</code> <code>s = g.exponential(2, (100000, 3)).sum(axis=1)</code> <code>round(float(s.mean()), 1)</code>	6.0
P($X < 6$)	Probabilidad de atender a 3 antes de 6 min.	<code>round(float(d.cdf(6)), 4)</code>	0.5768
Erlang con Poisson	$P(X \leq 6) = P(\text{Poisson}(\beta \cdot 6 = 3) \geq 3)$.	<code>round(float(1 - stats.poisson.cdf(2, 3)), 4)</code>	0.5768
$\alpha = 1$ es exponencial	Misma densidad en $x = 2$ ($\text{scale} = 2$).	<code>print(round(float(stats.gamma.pdf(2, a=1, scale=2)), 4),</code> <code>round(float(stats.expon.pdf(2, scale=2)), 4))</code>	0.1839 0.1839
Percentil 90	Tiempo que cubre 90% de los casos.	<code>round(float(d.ppf(0.9)), 4)</code>	10.6446

Distribución normal

Tiene forma de campana, simétrica alrededor de la media μ , y su ancho lo da la desviación σ . Es el modelo típico de alturas, pesos y notas. Regla 68-95-99.7: $\mu \pm \sigma$, $\mu \pm 2\sigma$ y $\mu \pm 3\sigma$ cubren esos porcentajes.

RECUERDA Una campana: casi todos están cerca del centro y pocos en los extremos. σ es lo «ancha» que es la campana.

DATOS DE EJEMPLO

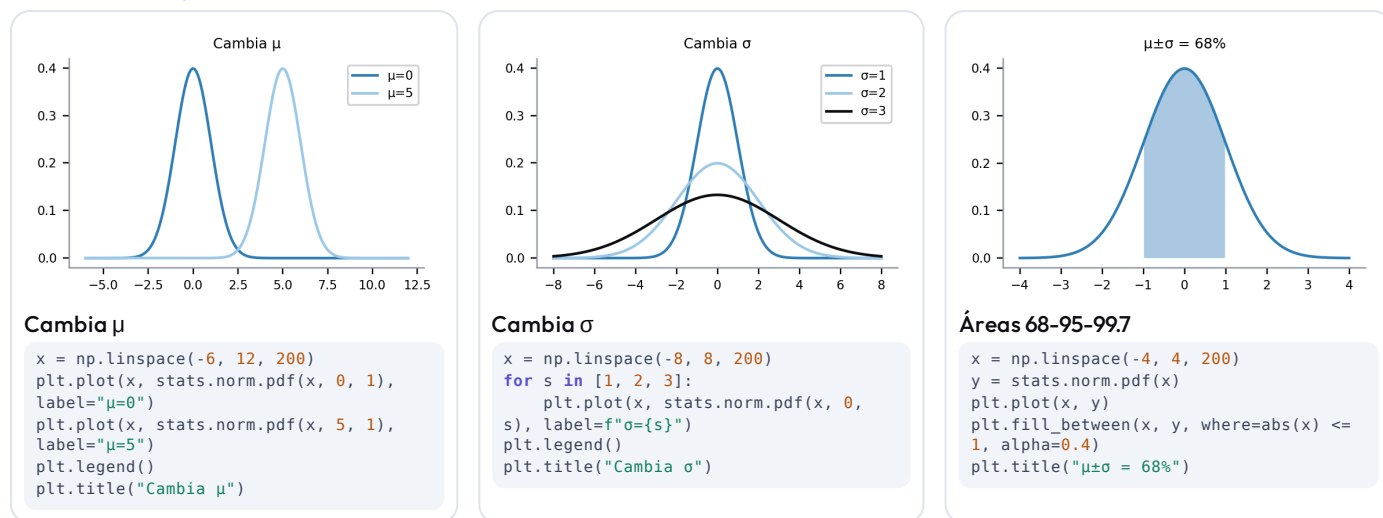
```
import numpy as np
from scipy import stats
from scipy.integrate import quad
d = stats.norm(165, 10)
```

MEDIDA	DESCRIPCIÓN	EJEMPLO	RESULTADO
Densidad en μ	$f(\mu) = 1/(\sigma\sqrt{2\pi})$, el punto más alto.	<code>round(float(d.pdf(165)), 4)</code>	0.0399
Simetría	La mitad queda bajo la media.	<code>round(float(d.cdf(165)), 4)</code>	0.5
P($X < 175$)	cdf: área hasta 175 cm ($\mu + \sigma$).	<code>round(float(d.cdf(175)), 4)</code>	0.8413
$\mu \pm \sigma$: 68%	$P(155 < X < 175)$.	<code>round(float(d.cdf(175) - d.cdf(155)), 4)</code>	0.6827

MEDIDA	DESCRIPCIÓN	EJEMPLO	RESULTADO
$\mu \pm 2\sigma$: 95%	$P(145 < X < 185)$.	<code>round(float(d.cdf(185) - d.cdf(145)), 4)</code>	0.9545
$\mu \pm 3\sigma$: 99.7%	$P(135 < X < 195)$.	<code>round(float(d.cdf(195) - d.cdf(135)), 4)</code>	0.9973
$P(X > 180)$	Cola derecha: $1 - \text{cdf}$ (o sf).	<code>round(float(d.sf(180)), 4)</code>	0.0668
Percentil 95	Altura que supera al 95% del grupo.	<code>round(float(d.ppf(0.95)), 4)</code>	181.4485

Ejemplo: alturas de estudiantes con $\mu = 165 \text{ cm}$ y $\sigma = 10 \text{ cm}$.

Gráficos de ejemplo



TEMA 44

Normal estándar y estandarización

La normal estándar Z tiene $\mu = 0$ y $\sigma = 1$. Cualquier normal se convierte a Z con $Z = (X - \mu) / \sigma$, que dice cuántas desviaciones está X de la media. Así se usa una sola tabla Z .

RECUERDA Z es como pasar todo a la misma moneda: una nota de 85 y una altura de 180 se comparan en «desviaciones».

DATOS DE EJEMPLO

```

import numpy as np
from scipy import stats
from scipy.integrate import quad

```

IDEA	DESCRIPCIÓN	EJEMPLO	RESULTADO
$Z = (X - \mu) / \sigma$	Nota 85 con $\mu = 70$ y $\sigma = 10$.	<code>z = (85 - 70) / 10</code> <code>print(z)</code>	1.5
Tabla Z	$P(Z < 1.5) = \text{cdf}$ de la normal estándar.	<code>round(float(stats.norm.cdf(1.5)), 4)</code>	0.9332
Sin estandarizar	Mismo resultado usando μ y σ directo.	<code>round(float(stats.norm.cdf(85, 70, 10)), 4)</code>	0.9332
$P(Z < 1.96)$	El valor clásico: 97.5%.	<code>round(float(stats.norm.cdf(1.96)), 4)</code>	0.975
$P(-1.96 < Z < 1.96)$	El 95% central.	<code>round(float(stats.norm.cdf(1.96) - stats.norm.cdf(-1.96)), 4)</code>	0.95
Simetría	$P(Z < -1) = 1 - P(Z < 1)$.	<code>print(round(float(stats.norm.cdf(-1)), 4), round(float(1 - stats.norm.cdf(1)), 4))</code>	0.1587 0.1587

IDEA	DESCRIPCIÓN	EJEMPLO	RESULTADO
Altura 180	$\mu = 170, \sigma = 8: z = 1.25, P(X > 180).$	<code>z = (180 - 170) / 8 round(float(1 - stats.norm.cdf(z)), 4)</code>	0.1056
Despejar $X = \mu + z \cdot \sigma$	Nota del percentil 90 ($z = 1.2816$).	<code>z = stats.norm.ppf(0.9) round(float(70 + z * 10), 4)</code>	82.8155

TEMA 45

Aproximación normal a la binomial

Si n es grande, la binomial se parece a una normal con $\mu = np$ y $\sigma = \sqrt{npq}$. Se usa cuando $np \geq 5$ y $nq \geq 5$. Como la binomial es discreta, se aplica corrección por continuidad: ± 0.5 .

RECUERDA Es cambiar escalones por una rampa suave. Al pasar de escalones a rampa, el 0.5 evita perder medio escalón.

DATOS DE EJEMPLO

```
import numpy as np
from scipy import stats
from scipy.integrate import quad
n, p = 50, 0.4
mu = n * p
sd = (n * p * (1 - p)) ** 0.5
```

PASO	DESCRIPCIÓN	EJEMPLO	RESULTADO
Condiciones	$np \geq 5$ y $nq \geq 5$ ($n = 50, p = 0.4$).	<code>print(n * p, n * (1 - p))</code>	20.0 30.0
μ y σ	$\mu = np$ y $\sigma = \sqrt{npq}$.	<code>print(mu, round(sd, 4))</code>	20.0 3.4641
Exacto $P(X \leq 18)$	Binomial de verdad.	<code>round(float(stats.binom.cdf(18, n, p)), 4)</code>	0.3356
Aproximado $P(X \leq 18)$	Normal con corrección: se usa 18.5.	<code>round(float(stats.norm.cdf(18.5, mu, sd)), 4)</code>	0.3325
Sin corrección	Usar 18 da peor aproximación.	<code>round(float(stats.norm.cdf(18, mu, sd)), 4)</code>	0.2819
Exacto $P(X = 20)$	pmf de la binomial.	<code>round(float(stats.binom.pmf(20, n, p)), 4)</code>	0.1146
Aproximado $P(X = 20)$	Área entre 19.5 y 20.5.	<code>round(float(stats.norm.cdf(20.5, mu, sd) - stats.norm.cdf(19.5, mu, sd)), 4)</code>	0.1148
Reglas de corrección	$X \leq k \rightarrow k + 0.5$ $X < k \rightarrow k - 0.5$ $X \geq k \rightarrow k - 0.5$ $X > k \rightarrow k + 0.5$	$P(X \geq 25) = P(\text{Normal} \geq 24.5)$	Sale de restar o sumar 0.5 al borde del intervalo

Ejemplo: de 50 estudiantes, cada uno aprueba con probabilidad 0.4.

Distribución Weibull

Modela el tiempo de vida de equipos. Tiene forma c y escala (vida típica). Si $c > 1$ la falla aumenta con el desgaste, si $c = 1$ es exponencial y si $c < 1$ hay fallas tempranas. En scipy: `weibull_min(c, scale)`.

RECUERDA c dice cómo envejece el foco: $c > 1$ se desgasta, $c = 1$ falla al azar, $c < 1$ los defectuosos fallan pronto.

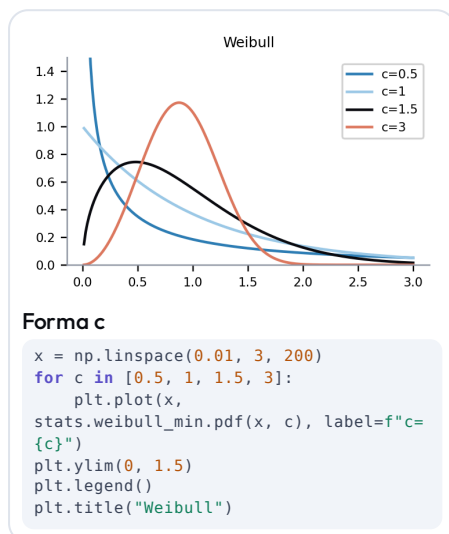
DATOS DE EJEMPLO

```
import numpy as np
from scipy import stats
from scipy.integrate import quad
import math
d = stats.weibull_min(c=1.5, scale=1000)
```

MEDIDA	DESCRIPCIÓN	EJEMPLO	RESULTADO
P(falla antes de 500 h)	$\text{cdf} = 1 - \exp(-(t/\text{scale})^c)$, con $c = 1.5$ y $\text{scale} = 1000$.	<code>round(float(d.cdf(500)), 4)</code>	0.2978
Con la fórmula	Mismo valor calculado a mano.	<code>round(1 - math.exp(-(500 / 1000)** 1.5), 4)</code>	0.2978
P(dura más de 1000 h)	$\exp(-1)$ cuando $t = \text{scale}$.	<code>round(float(d.sf(1000)), 4)</code>	0.3679
Media	$\text{scale} \cdot \text{gamma}(1 + 1/c)$ horas.	<code>print(round(float(d.mean()), 2), round(1000 * math.gamma(1 + 1 / 1.5), 2))</code>	902.75 902.75
Mediana	La mitad de los focos dura menos.	<code>round(float(d.ppf(0.5)), 2)</code>	783.22
Garantía 10% de fallas	Tiempo en que falla el 10%.	<code>round(float(d.ppf(0.10)), 2)</code>	223.08
Tasa de falla	pdf/sf aumenta con el tiempo ($c > 1$).	<code>print(round(float(d.pdf(500) / d.sf(500)), 5), round(float(d.pdf(1000) / d.sf(1000)), 5))</code>	0.00106 0.0015
$c = 1$ es exponencial	Mismo cdf en $t = 500$ ($\text{scale} = 1000$).	<code>print(round(float(stats.weibull_min.cdf(500, 1, scale=1000)), 4), round(float(stats.expon.cdf(500, scale=1000)), 4))</code>	0.3935 0.3935

Ejemplo: vida útil de focos en horas con $c = 1.5$ y $\text{scale} = 1000$.

Gráficos de ejemplo



Cómo elegir el modelo continuo

Primero pregunta qué mides y cómo se comporta. Si es un tiempo de espera, una vida útil o una medida, mira si es parejo, si hay desgaste o si es simétrico. Esa pista indica el modelo.

RECUERDA Mira el verbo: «esperar» sugiere exponencial o gamma; «medir» sugiere normal; «cualquier instante» sugiere uniforme.

SITUACIÓN	DESCRIPCIÓN	EJEMPLO	RESULTADO
Todo valor igual de probable	Llegada al azar dentro de un intervalo → Uniforme.	<code>stats.uniform(loc=a, scale=b - a)</code>	Uniforme media $(a + b)/2$
Tiempo entre eventos al azar	Tiempo hasta el primer evento → Exponencial.	<code>stats.expon(scale=1 / lam)</code>	Exponencial media $1/\lambda$
Tiempo hasta el evento α	Suma de α exponenciales → Gamma (Erlang).	<code>stats.gamma(a=alfa, scale=1 / beta)</code>	Gamma media α/β
Alturas, notas, pesos	Campana simétrica alrededor de la media → Normal.	<code>stats.norm(mu, sigma)</code>	Normal Usa $Z = (X - \mu)/\sigma$
Vida útil con desgaste	Focos o equipos que envejecen → Weibull.	<code>stats.weibull_min(c, scale=s)</code>	Weibull $c > 1$: aumenta la falla
Muchos ensayos sí/no	Binomial con n grande y $np, nq \geq 5$ → Normal.	<code>stats.norm(n * p, (n * p * q) ** 0.5)</code>	Normal aproximada Usa corrección ± 0.5
¿Contar o medir?	Contar enteros → discreta. Medir en un intervalo → continua.	discreta o continua	Contar: pmf y sumas Medir: densidad e integrales

Después de elegir el modelo, usa *cdf* para áreas, *ppf* para percentiles y *mean()* para el promedio.

Vectores aleatorios

Dos o más variables aleatorias al mismo tiempo y cómo se relacionan.

TEMA 48

Vector aleatorio y distribución conjunta discreta

Un vector aleatorio (X, Y) mide dos variables a la vez en el mismo experimento. Su distribución conjunta es una tabla con $p(x, y) = P(X = x, Y = y)$. Todas las celdas son ≥ 0 y la tabla completa suma 1.

RECUERDA La tabla conjunta es como un horario de clases: fila = una variable, columna = la otra y cada celda dice qué tan probable es esa pareja.

DATOS DE EJEMPLO

```
import numpy as np
p = np.array([[0.02, 0.06, 0.12],
              [0.10, 0.20, 0.20],
              [0.18, 0.04, 0.08]])
x = np.array([0, 1, 2])
y = np.array([1, 2, 3])
import pandas as pd
```

IDEA	DESCRIPCIÓN	EJEMPLO	RESULTADO																
p (matriz)	Tabla conjunta: fila = valor de X, columna = valor de Y.	p	[[0.02 0.06 0.12] [0.1 0.2 0.2] [0.18 0.04 0.08]]																
pd.DataFrame()	La misma tabla con los valores de X y Y como títulos.	pd.DataFrame(p, index=x, columns=y)	<table> <tr> <th></th><th>1</th><th>2</th><th>3</th></tr> <tr> <th>0</th><td>0.02</td><td>0.06</td><td>0.12</td></tr> <tr> <th>1</th><td>0.10</td><td>0.20</td><td>0.20</td></tr> <tr> <th>2</th><td>0.18</td><td>0.04</td><td>0.08</td></tr> </table>		1	2	3	0	0.02	0.06	0.12	1	0.10	0.20	0.20	2	0.18	0.04	0.08
	1	2	3																
0	0.02	0.06	0.12																
1	0.10	0.20	0.20																
2	0.18	0.04	0.08																
Suma total	$\sum \sum p(x, y) = 1$. Si no da 1, no es una distribución.	round(p.sum(), 4)	1.0																
P(X=1, Y=2)	Una celda: fila de X = 1, columna de Y = 2.	p[1, 1]	0.2																
P(X=0, Y=3)	Otra celda de la tabla.	p[0, 2]	0.12																
P(X ≥ 1, Y ≥ 2)	Suma de las celdas que cumplen las dos condiciones.	round(p[1:, 1:].sum(), 4)	0.52																
P(X + Y = 3)	Suma de las parejas cuya suma es 3.	m = np.add.outer(x, y) == 3 round(p[m].sum(), 4)	0.5																
¿Es válida?	Ninguna celda negativa y suma total igual a 1.	a = (p >= 0).all() b = np.isclose(p.sum(), 1) print(a, b)	True True																

X = materias reprobadas (0, 1, 2) y Y = horas de estudio al día (1, 2, 3). Los índices de numpy empiezan en 0.

Distribuciones marginales

La marginal de una variable es su distribución sola, sin importar la otra. Se obtiene sumando la tabla: $pX(x) = \sum p(x,y)$ sumando por filas y $pY(y) = \sum p(x,y)$ sumando por columnas.

RECUERDA Marginal = los totales en el borde de la tabla. Sumas la fila y miras solo a X; sumas la columna y miras solo a Y.

DATOS DE EJEMPLO

```
import numpy as np
p = np.array([[0.02, 0.06, 0.12],
              [0.10, 0.20, 0.20],
              [0.18, 0.04, 0.08]])
x = np.array([0, 1, 2])
y = np.array([1, 2, 3])
pX = p.sum(axis=1)
pY = p.sum(axis=0)
```

MEDIDA	DESCRIPCIÓN	EJEMPLO	RESULTADO
pX(x)	Suma de cada fila (axis=1): distribución de X.	<code>np.round(p.sum(axis=1), 4)</code>	[0.2 0.5 0.3]
pY(y)	Suma de cada columna (axis=0): distribución de Y.	<code>np.round(p.sum(axis=0), 4)</code>	[0.3 0.3 0.4]
Cada marginal suma 1	Son distribuciones completas.	<code>a, b = pX.sum(), pY.sum()</code> <code>print(round(a, 4), round(b, 4))</code>	1.0 1.0
P(X = 2)	Un valor de la marginal de X.	<code>round(pX[2], 4)</code>	0.3
P(Y ≥ 2)	Suma de la marginal de Y desde el valor 2.	<code>round(pY[1:].sum(), 4)</code>	0.7
E[X]	$E[X] = \sum x \cdot pX(x)$, usando la marginal.	<code>round((x * pX).sum(), 4)</code>	1.1
Var(X)	$Var(X) = \sum (x - E[X])^2 \cdot pX(x)$.	<code>m = (x * pX).sum()</code> <code>round(((x - m) ** 2 * pX).sum(), 4)</code>	0.49

Distribuciones condicionales

La condicional dice cómo se reparte X cuando ya sabemos el valor de Y: $p(x|y) = p(x,y) / pY(y)$. Es la columna de la tabla dividida entre el total de esa columna, y siempre suma 1.

RECUERDA Condicional = quedarte solo con una columna y reajustarla para que sume 100%. Es «dado que ya sé que Y = y».

DATOS DE EJEMPLO

```
import numpy as np
p = np.array([[0.02, 0.06, 0.12],
              [0.10, 0.20, 0.20],
              [0.18, 0.04, 0.08]])
x = np.array([0, 1, 2])
y = np.array([1, 2, 3])
pX = p.sum(axis=1)
pY = p.sum(axis=0)
```

MEDIDA	DESCRIPCIÓN	EJEMPLO	RESULTADO
p(x Y=2)	La columna de Y = 2 dividida entre pY(2).	<code>np.round(p[:, 1] / pY[1], 3)</code>	[0.2 0.667 0.133]
p(y X=0)	La fila de X = 0 dividida entre pX(0).	<code>np.round(p[0] / pX[0], 3)</code>	[0.1 0.3 0.6]
P(X=2 Y=1)	$p(2,1) / pY(1)$.	<code>round(p[2, 0] / pY[0], 4)</code>	0.6

MEDIDA	DESCRIPCIÓN	EJEMPLO	RESULTADO
Suma 1	Una condicional es una distribución completa.	<code>round((p[:, 1] / pY[1]).sum(), 4)</code>	1.0
Todas las $p(x y)$	Cada columna dividida entre su total.	<code>np.round(p / pY, 3)</code>	<code>[[0.067 0.2 0.3]</code> <code>[0.333 0.667 0.5]</code> <code>[0.6 0.133 0.2]]</code>
$E[X Y=3]$	$E[X y] = \sum x \cdot p(x y)$.	<code>round((x * p[:, 2] / pY[2]).sum(), 4)</code>	0.9
$E[Y X=2]$	$E[Y x] = \sum y \cdot p(y x)$.	<code>round((y * p[2] / pX[2]).sum(), 4)</code>	1.6667
Regla del producto	$p(x,y) = p(x y) \cdot pY(y)$.	<code>np.allclose((p / pY) * pY, p)</code>	True

TEMA 51

Independencia de variables aleatorias

X y Y son independientes si $p(x,y) = pX(x) \cdot pY(y)$ en TODAS las celdas. Entonces saber una no cambia nada sobre la otra. Basta una celda que no cumpla para que sean dependientes.

RECUERDA Independientes = lanzar dos monedas distintas: lo que sale en una no afecta a la otra.

DATOS DE EJEMPLO

```
import numpy as np
p = np.array([[0.02, 0.06, 0.12],
              [0.10, 0.20, 0.20],
              [0.18, 0.04, 0.08]])
x = np.array([0, 1, 2])
y = np.array([1, 2, 3])
pX = p.sum(axis=1)
pY = p.sum(axis=0)
```

PRUEBA	DESCRIPCIÓN	EJEMPLO	RESULTADO
<code>np.outer(pX, pY)</code>	Tabla que tendríamos si fueran independientes.	<code>np.round(np.outer(pX, pY), 3)</code>	<code>[[0.06 0.06 0.08]</code> <code>[0.15 0.15 0.2]</code> <code>[0.09 0.09 0.12]]</code>
Una celda	Real $p(0,1)$ frente a $pX(0) \cdot pY(1)$.	<code>print(p[0, 0], round(pX[0] * pY[0], 4))</code>	0.02 0.06
<code>np.allclose()</code>	¿Son iguales todas las celdas? Falso: dependientes.	<code>np.allclose(p, np.outer(pX, pY))</code>	False
Condicional vs marginal	Si fueran independientes, $p(x y) = pX(x)$.	<code>np.allclose(p[:, 0] / pY[0], pX)</code>	False
Tabla independiente	Construida como producto de marginales.	<code>q = np.outer([.5, .3, .2], [.4, .6])</code> <code>a, b = q.sum(1), q.sum(0)</code> <code>np.allclose(q, np.outer(a, b))</code>	True
Un cero estorba	Si $p(x,y) = 0$ pero las marginales no, hay dependencia.	<code>z = np.array([[0.5, 0], [0, 0.5]])</code> <code>a, b = z.sum(1), z.sum(0)</code> <code>np.allclose(z, np.outer(a, b))</code>	False
Fórmula	$p(x,y) = pX(x) \cdot pY(y)$ para todo par (x, y) .	<code>pX(x) * pY(y)</code>	Se revisa celda por celda

Esperanza de funciones de X e Y

Para una función $g(X,Y)$ su esperanza es $E[g(X,Y)] = \sum \sum g(x,y) \cdot p(x,y)$. Se calcula el valor de g en cada celda, se multiplica por su probabilidad y se suma todo.

RECUERDA Es un promedio ponderado: cada pareja aporta su valor multiplicado por qué tan probable es.

DATOS DE EJEMPLO

```
import numpy as np
p = np.array([[0.02, 0.06, 0.12],
              [0.10, 0.20, 0.20],
              [0.18, 0.04, 0.08]])
x = np.array([0, 1, 2])
y = np.array([1, 2, 3])
X, Y = np.meshgrid(x, y, indexing="ij")
```

FUNCIÓN G	DESCRIPCIÓN	EJEMPLO	RESULTADO
$E[XY]$	$\sum \sum x \cdot y \cdot p(x,y)$.	<code>round((X * Y * p).sum(), 4)</code>	2.1
$E[X + Y]$	$\sum \sum (x+y) \cdot p(x,y)$.	<code>round(((X + Y) * p).sum(), 4)</code>	3.2
$E[X^2]$	g depende solo de X : también se puede.	<code>round((X ** 2 * p).sum(), 4)</code>	1.7
$E[\text{máximo}(X, Y)]$	El mayor de los dos valores en cada celda.	<code>round((np.maximum(X, Y) * p).sum(), 4)</code>	2.28
$E[(X - Y)^2]$	Diferencia al cuadrado en cada celda.	<code>round(((X - Y) ** 2 * p).sum(), 4)</code>	2.6
$E[5X + 2Y]$	Costo en dólares: 5 por materia reprobada y 2 por hora.	<code>round(((5 * X + 2 * Y) * p).sum(), 4)</code>	9.7
Con dos ciclos for	Lo mismo que $E[XY]$, celda por celda.	<pre>t = 0 for i in range(3): for j in range(3): t += x[i] * y[j] * p[i, j] round(t, 4)</pre>	2.1
$E[XY] \neq E[X] \cdot E[Y]$	Solo serían iguales con covarianza 0.	<pre>ex = (X * p).sum() ey = (Y * p).sum() exy = (X * Y * p).sum() print(round(exy, 4), round(ex * ey, 4))</pre>	2.1 2.31

Covarianza

La covarianza mide si dos variables suben y bajan juntas: $\text{Cov}(X,Y) = E[XY] - E[X] \cdot E[Y]$. Positiva: suben juntas. Negativa: una sube y la otra baja. Cero: no hay relación lineal.

RECUERDA $\text{Cov} > 0$: más horas de gimnasio, más energía. $\text{Cov} < 0$: más horas de estudio, menos materias reprobadas.

DATOS DE EJEMPLO

```
import numpy as np
p = np.array([[0.02, 0.06, 0.12],
              [0.10, 0.20, 0.20],
              [0.18, 0.04, 0.08]])
x = np.array([0, 1, 2])
y = np.array([1, 2, 3])
X, Y = np.meshgrid(x, y, indexing="ij")
mx, my = (X * p).sum(), (Y * p).sum()
```

MEDIDA	DESCRIPCIÓN	EJEMPLO	RESULTADO
$E[XY]$	Esperanza del producto, con la tabla.	<code>round((X * Y * p).sum(), 4)</code>	2.1
$E[X]$ y $E[Y]$	Las medias de cada variable.	<code>print(round(mx, 4), round(my, 4))</code>	1.1 2.1
$\text{Cov}(X,Y)$	$E[XY] - E[X] \cdot E[Y]$.	<code>round((X * Y * p).sum() - mx * my, 4)</code>	-0.21
Forma centrada	$\text{Cov} = E[(X - E[X])(Y - E[Y])]$; mismo resultado.	<code>round(((X - mx) * (Y - my) * p).sum(), 4)</code>	-0.21
Signo	El signo indica la dirección de la relación.	<code>c = (X * Y * p).sum() - mx * my</code> <code>print("negativa" if c < 0 else "positiva")</code>	negativa
$\text{Cov}(X,X)$	La covarianza de X consigo misma es $\text{Var}(X)$.	<code>round(((X - mx) ** 2 * p).sum(), 4)</code>	0.49
Si son independientes	$\text{Cov} = 0$. Con $q = pX \cdot pY$ da cero.	<code>q = np.outer(p.sum(1), p.sum(0))</code> <code>c = (X * Y * q).sum() - mx * my</code> <code>round(abs(c), 4)</code>	0.0
$\text{Cov}(2X, 3Y)$	$\text{Cov}(aX, bY) = a \cdot b \cdot \text{Cov}(X,Y)$.	<code>c = (X * Y * p).sum() - mx * my</code> <code>round(6 * c, 4)</code>	-1.26

La covarianza depende de las unidades; por eso se usa la correlación para comparar.

Correlación

La correlación es la covarianza sin unidades: $\rho = \text{Cov}(X,Y) / (\sigma_X \cdot \sigma_Y)$. Siempre está entre -1 y 1. Cerca de 1 o -1 hay una relación lineal fuerte; cerca de 0, casi nada lineal.

RECUERDA Independientes $\Rightarrow \rho = 0$, pero $\rho = 0$ NO implica independientes: puede haber una relación curva que ρ no ve.

DATOS DE EJEMPLO

```
import numpy as np
p = np.array([[0.02, 0.06, 0.12],
              [0.10, 0.20, 0.20],
              [0.18, 0.04, 0.08]])
x = np.array([0, 1, 2])
y = np.array([1, 2, 3])
X, Y = np.meshgrid(x, y, indexing="ij")
mx, my = (X * p).sum(), (Y * p).sum()
vx = ((X - mx) ** 2 * p).sum()
vy = ((Y - my) ** 2 * p).sum()
cxy = ((X - mx) * (Y - my) * p).sum()
a, b = X.ravel(), Y.ravel()
pX = p.sum(axis=1)
pY = p.sum(axis=0)
```

MEDIDA	DESCRIPCIÓN	EJEMPLO	RESULTADO
σ_X	$\sigma_X = \sqrt{\text{Var}(X)}$.	<code>round(vx ** 0.5, 4)</code>	0.7
σ_Y	$\sigma_Y = \sqrt{\text{Var}(Y)}$.	<code>round(vy ** 0.5, 4)</code>	0.8307
ρ	$\rho = \text{Cov} / (\sigma_X \cdot \sigma_Y)$.	<code>round(cxy / (vx * vy) ** 0.5, 4)</code>	-0.3612
Siempre entre -1 y 1	Comprobación con el valor de ρ .	<code>r = cxy / (vx * vy) ** 0.5</code> <code>-1 <= r <= 1</code>	True
No depende de la escala	Multiplicar X por 2 y Y por 3 no cambia ρ .	<code>sx, sy = vx ** 0.5, vy ** 0.5</code> <code>round(6 * cxy / (2 * sx * 3 * sy), 4)</code>	-0.3612
$\rho = 1$	Recta creciente exacta: $Y = X + 1$ (tabla diagonal).	<code>w = np.diag([0.2, 0.5, 0.3]).ravel()</code> <code>C = np.cov(a, b, aweights=w)</code> <code>s = np.sqrt(np.diag(C))</code> <code>round(C[0, 1] / (s[0] * s[1]), 4)</code>	1.0
Independientes $\rightarrow \rho = 0$	La tabla $q = pX \cdot pY$ tiene correlación 0.	<code>w = np.outer(pX, pY).ravel()</code> <code>C = np.cov(a, b, aweights=w)</code> <code>s = np.sqrt(np.diag(C))</code> <code>round(C[0, 1] / (s[0] * s[1]), 4) + 0.0</code>	0.0
$\rho = 0$ sin independencia	$V = U^2$ depende de U, pero su covarianza es 0.	<code>u = np.array([-1, 0, 1])</code> <code>pu = np.array([0.25, 0.5, 0.25])</code> <code>E = lambda g: (g * pu).sum()</code> <code>print(E(u * u**2) - E(u) * E(u**2))</code>	0.0

Combinaciones lineales

Para $aX + bY$: $E[aX + bY] = a \cdot E[X] + b \cdot E[Y]$ (siempre, sean o no independientes). Y $\text{Var}(aX + bY) = a^2 \cdot \text{Var}(X) + b^2 \cdot \text{Var}(Y) + 2ab \cdot \text{Cov}(X, Y)$. Si son independientes, el último término desaparece.

RECUERDA La media se reparte sin condiciones. La varianza no: si X y Y se mueven juntas hay que añadir el término $2ab \cdot \text{Cov}$.

DATOS DE EJEMPLO

```
import numpy as np
p = np.array([[0.02, 0.06, 0.12],
              [0.10, 0.20, 0.20],
              [0.18, 0.04, 0.08]])
x = np.array([0, 1, 2])
y = np.array([1, 2, 3])
X, Y = np.meshgrid(x, y, indexing="ij")
mx, my = (X * p).sum(), (Y * p).sum()
vx = ((X - mx) ** 2 * p).sum()
vy = ((Y - my) ** 2 * p).sum()
cxy = ((X - mx) * (Y - my) * p).sum()
```

CÁLCULO	DESCRIPCIÓN	EJEMPLO	RESULTADO
E[2X + 3Y] directo	Con la tabla, celda por celda.	<code>round(((2 * X + 3 * Y) * p).sum(), 4)</code>	8.5
2E[X] + 3E[Y]	Con la fórmula: da lo mismo.	<code>round(2 * mx + 3 * my, 4)</code>	8.5
Var(2X + 3Y) directo	Se calcula $Z = 2X + 3Y$ y su varianza.	<code>Z = 2 * X + 3 * Y</code> <code>mz = (Z * p).sum()</code> <code>round(((Z - mz) ** 2 * p).sum(), 4)</code>	5.65
Var(2X + 3Y) fórmula	$4 \cdot \text{Var} X + 9 \cdot \text{Var} Y + 2 \cdot 2 \cdot 3 \cdot \text{Cov}$.	<code>round(4 * vx + 9 * vy + 12 * cxy, 4)</code>	5.65
Var(X + Y)	$\text{Var} X + \text{Var} Y + 2 \cdot \text{Cov}$.	<code>round(vx + vy + 2 * cxy, 4)</code>	0.76
Var(X - Y)	$\text{Var} X + \text{Var} Y - 2 \cdot \text{Cov}$ ($a = 1, b = -1$).	<code>round(vx + vy - 2 * cxy, 4)</code>	1.6
Error: olvidar Cov	Sin el término $2ab \cdot \text{Cov}$ sale otro valor.	<code>mal = vx + vy</code> <code>bien = mal + 2 * cxy</code> <code>print(round(mal, 4), round(bien, 4))</code>	1.18 0.76
Promedio (X + Y) / 2	$\text{Var} = \frac{1}{4} \text{Var} X + \frac{1}{4} \text{Var} Y + \frac{1}{2} \text{Cov}$.	<code>round(vx / 4 + vy / 4 + cxy / 2, 4)</code>	0.19

Distribución conjunta continua

Si X y Y son continuas se usa una densidad conjunta $f(x, y) \geq 0$. Las probabilidades son volúmenes: $P((X, Y) \in A) = \iint_A f(x, y) dx dy$, y el volumen total vale 1. La marginal se obtiene integrando la otra variable.

RECUERDA En el caso discreto sumas celdas; en el continuo integras. Es la misma idea con integrales en lugar de sumas.

DATOS DE EJEMPLO

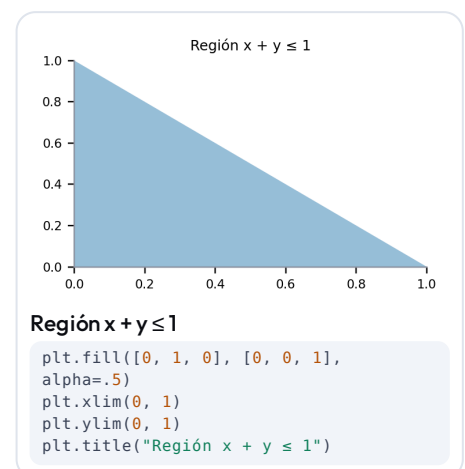
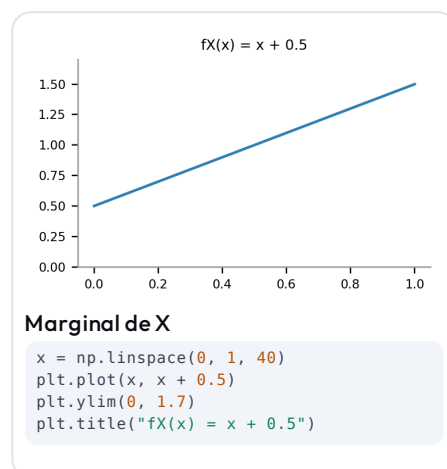
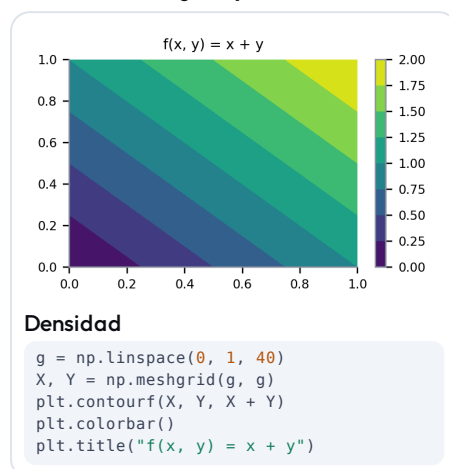
```
import numpy as np
from scipy import integrate
f = lambda y, x: x + y
```

CÁLCULO	DESCRIPCIÓN	EJEMPLO	RESULTADO
Volumen total	$\iint f = 1$ sobre el cuadrado $[0, 1] \times [0, 1]$.	<code>r = integrate.dblquad(f, 0, 1, 0, 1)</code> <code>round(r[0], 4)</code>	1.0

CÁLCULO	DESCRIPCIÓN	EJEMPLO	RESULTADO
$P(X \leq 0.5, Y \leq 0.5)$	Integral doble sobre el cuadrado inferior.	<pre>r = integrate.dblquad(f, 0, .5, 0, .5) round(r[0], 4)</pre>	0.125
$P(X + Y \leq 1)$	Región triangular: y va de 0 a 1-x.	<pre>lim = lambda x: 1 - x r = integrate.dblquad(f, 0, 1, 0, lim) round(r[0], 4)</pre>	0.3333
$f_X(x)$ en $x = 0.3$	Marginal: integrar $f(x,y)$ respecto a y.	<pre>g = lambda y: 0.3 + y round(integrate.quad(g, 0, 1)[0], 4)</pre>	0.8
$P(X \leq 0.5)$	Integrar y de 0 a 1 y x de 0 a 0.5.	<pre>r = integrate.dblquad(f, 0, .5, 0, 1) round(r[0], 4)</pre>	0.375
$E[X]$	$\iint x \cdot f(x,y) dx dy$.	<pre>h = lambda y, x: x * (x + y) r = integrate.dblquad(h, 0, 1, 0, 1) round(r[0], 4)</pre>	0.5833
$E[XY]$	$\iint x \cdot y \cdot f(x,y) dx dy$.	<pre>h = lambda y, x: x * y * (x + y) r = integrate.dblquad(h, 0, 1, 0, 1) round(r[0], 4)</pre>	0.3333
¿Independientes?	En (0,0): f vale 0, pero $f_X \cdot f_Y$ vale 0.25.	<pre>fx = lambda t: t + 0.5 print(f(0, 0), fx(0) * fx(0))</pre>	0 0.25

Densidad de ejemplo: $f(x,y) = x + y$ en el cuadrado unitario. `dblquad(f, x1, x2, y1, y2)` pide f con el orden (y, x).

Gráficos de ejemplo



Distribución multinomial

Generaliza la binomial a más de dos categorías. Se hacen n pruebas independientes y cada una cae en una de k categorías con probabilidades $p_1, \dots, p_k$. Cuenta cuántas pruebas caen en cada categoría.

RECUERDA Binomial = éxito o fracaso. Multinomial = elegir entre varias opciones, como el sabor de helado preferido de cada amigo.

DATOS DE EJEMPLO

```
import numpy as np
from scipy import stats
n = 6
pr = [0.5, 0.3, 0.2]
```

CÁLCULO	DESCRIPCIÓN	EJEMPLO	RESULTADO
<code>pmf([3, 2, 1])</code>	P(3 en Computación, 2 en Electrónica, 1 en Mecánica).	<code>c = [3, 2, 1]</code> <code>q = stats.multinomial.pmf(c, n, pr)</code> <code>round(q, 4)</code>	0.135
<code>pmf([6, 0, 0])</code>	Los 6 eligen Computación: 0.5^6 .	<code>c = [6, 0, 0]</code> <code>q = stats.multinomial.pmf(c, n, pr)</code> <code>round(q, 4)</code>	0.0156
<code>pmf([2, 2, 2])</code>	Reparto parejo entre las tres carreras.	<code>c = [2, 2, 2]</code> <code>q = stats.multinomial.pmf(c, n, pr)</code> <code>round(q, 4)</code>	0.081
No suman n	Los conteos deben sumar $n = 6$; si no, $P = 0$.	<code>c = [3, 3, 3]</code> <code>q = stats.multinomial.pmf(c, n, pr)</code> <code>round(q, 4)</code>	0.0
Media	$E[X_i] = n \cdot p_i$ para cada categoría.	<code>stats.multinomial.mean(n, pr)</code>	[3. 1.8 1.2]
Varianza	$\text{Var}(X_i) = n \cdot p_i \cdot (1 - p_i)$.	<code>np.diag(stats.multinomial.cov(n, pr))</code>	[1.5 1.26 0.96]
<code>Cov(X1, X2)</code>	$\text{Cov}(X_i, X_j) = -n \cdot p_i \cdot p_j$: si una sube, otra baja.	<code>q = stats.multinomial.cov(n, pr)</code> <code>round(q[0, 1], 4)</code>	-0.9
Marginal binomial	Cada categoría sola es Binomial(n, p_i).	<code>q = stats.binom.pmf(3, n, 0.5)</code> <code>round(q, 4)</code>	0.3125

6 estudiantes eligen carrera: 50% Computación, 30% Electrónica, 20% Mecánica.

Distribución normal bivariada

Es la normal para dos variables: queda definida por el vector de medias μ y la matriz de covarianza Σ , con las varianzas en la diagonal. Sus curvas de nivel son elipses, inclinadas según el signo de ρ .

RECUERDA Una nube de puntos en forma de elipse: $\rho > 0$ la inclina hacia arriba, $\rho < 0$ hacia abajo y $\rho = 0$ la deja redonda o recta.

DATOS DE EJEMPLO

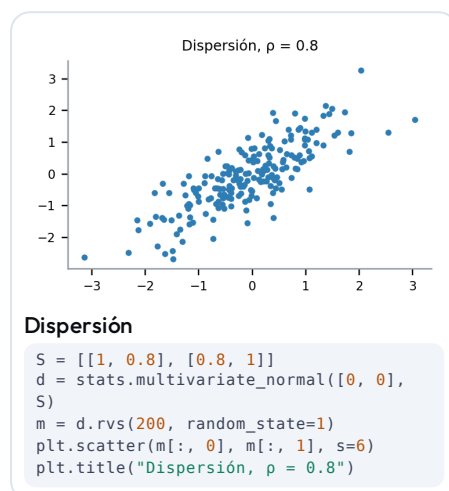
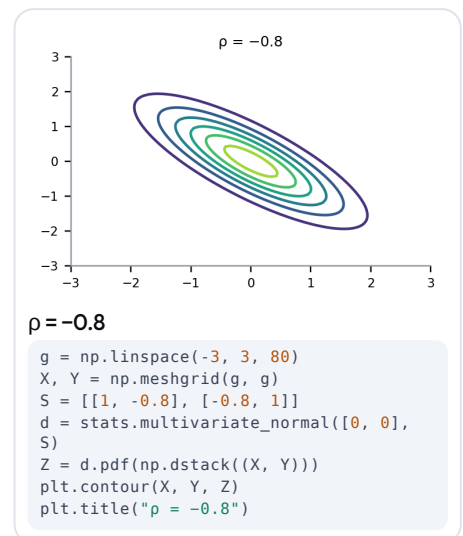
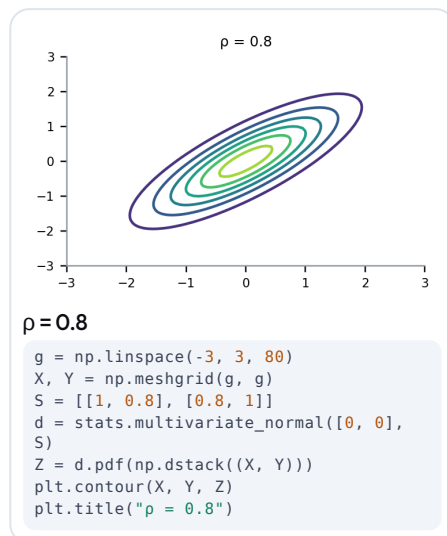
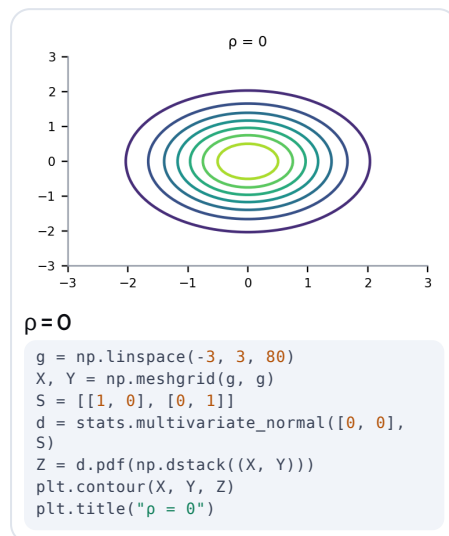
```
import numpy as np
from scipy import stats
mu = [70, 65]
S = [[100, 48], [48, 64]]
dist = stats.multivariate_normal(mu, S)
```

CÁLCULO	DESCRIPCIÓN	EJEMPLO	RESULTADO
Densidad en el centro	Valor de f en la media (el punto más alto).	<code>round(dist.pdf(mu), 6)</code>	0.002487

CÁLCULO	DESCRIPCIÓN	EJEMPLO	RESULTADO
Densidad en (80, 70)	<code>stats.multivariate_normal(mu, S).pdf(punto).</code>	<code>round(dist.pdf([80, 70]), 6)</code>	0.001508
ρ	$\rho = \text{Cov} / \sqrt{(\text{Var } X \cdot \text{Var } Y)}$, con la matriz S.	<code>v1, v2, c = 100, 64, 48</code> <code>round(c / (v1 * v2) ** 0.5, 2)</code>	0.6
$P(X \leq 70, Y \leq 65)$	Probabilidad acumulada conjunta con <code>cdf()</code> .	<code>round(dist.cdf([70, 65]), 3)</code>	0.352
Muestra simulada	3 parejas con semilla fija.	<code>m = dist.rvs(3, random_state=1)</code> <code>np.round(m, 1)</code>	<code>[[56.5 51.5]</code> <code>[78.4 63.6]</code> <code>[69.1 48.8]]</code>
Marginal de X	Cada una sola es normal: $X \sim N(70, 10^2)$.	<code>q = stats.norm.sf(80, 70, 10)</code> <code>round(q, 4)</code>	0.1587
$E[Y X=80]$	$\mu_Y + \rho \cdot (\sigma_Y / \sigma_X) \cdot (80 - \mu_X)$.	<code>ro, sx, sy = 0.6, 10, 8</code> <code>m = 65 + ro * sy / sx * (80 - 70)</code> <code>round(m, 2)</code>	69.8
Con $\rho = 0$	La densidad conjunta es el producto de las marginales.	<code>v = [100, 64]</code> <code>d0 = stats.multivariate_normal(mu, v)</code> <code>a = stats.norm.pdf(70, 70, 10)</code> <code>b = stats.norm.pdf(65, 65, 8)</code> <code>print(np.round([d0.pdf(mu), a * b], 6))</code>	<code>[0.001989 0.001989]</code>

$X = \text{nota de Matemáticas (media 70, } \sigma = 10)$, $Y = \text{nota de Física (media 65, } \sigma = 8)$ y $\rho = 0.6$.

Gráficos de ejemplo



Convergencia y distribuciones muestrales

Del azar de una muestra a las bases de la estadística inferencial.

TEMA 59

Muestra aleatoria y estadístico

Una muestra aleatoria simple es un grupo de la población elegido al azar, donde todos tienen la misma opción de salir. Un parámetro es el valor de la población (μ) y un estadístico es el valor calculado con la muestra ($\bar{x}$).

RECUERDA Parámetro = el promedio de TODO ESPOL (casi imposible de medir). Estadístico = el promedio de los 30 que sí encuestaste.

DATOS DE EJEMPLO

```
import numpy as np
from scipy import stats
rng = np.random.default_rng(1)
pob = np.round(np.clip(rng.normal(7, 1.5, 1000), 0, 10), 1)
```

CONCEPTO	DESCRIPCIÓN	EJEMPLO	RESULTADO
Población	Todos los datos. Su promedio es el parámetro μ .	<code>print(len(pob))</code> <code>print(round(pob.mean(), 3))</code>	1000 6.908
<code>rng.choice()</code>	Saca una muestra al azar sin repetir personas.	<code>m = rng.choice(pob, 30,</code> <code>replace=False)</code> <code>print(len(m))</code>	30
Estadístico $\bar{x}$	media muestral = suma de la muestra / n.	<code>m = rng.choice(pob, 30,</code> <code>replace=False)</code> <code>round(m.mean(), 3)</code>	7.127
Otra muestra	Otra muestra da otro estadístico: $\bar{x}$ varía.	<code>a = rng.choice(pob, 30,</code> <code>replace=False)</code> <code>b = rng.choice(pob, 30,</code> <code>replace=False)</code> <code>print(a.mean().round(3))</code> <code>print(b.mean().round(3))</code>	7.127 7.103
Error de muestreo	$\text{error} = \bar{x} - \mu$ (qué tanto nos equivocamos).	<code>m = rng.choice(pob, 30,</code> <code>replace=False)</code> <code>round(m.mean() - pob.mean(), 3)</code>	0.219
Proporción muestral	Con nota ≥ 7 : población y luego muestra ($\hat{p}$).	<code>m = rng.choice(pob, 30,</code> <code>replace=False)</code> <code>print((pob >= 7).mean().round(3))</code> <code>print((m >= 7).mean().round(3))</code>	0.5 0.6
Con reemplazo	<code>replace=True</code> : una persona puede salir dos veces.	<code>m = rng.choice(pob, 5,</code> <code>replace=True)</code> <code>print(len(m))</code>	5

Distribución muestral de la media

Si sacas muchas muestras de tamaño n y calculas $\bar{x}$ en cada una, esos $\bar{x}$ forman una distribución. Su centro es μ y su dispersión es el error estándar $\sigma/\sqrt{n}$: más datos, menos variación.

RECUERDA Un tirador con 25 balas por intento: sus promedios se agrupan más cerca del centro que los de un tirador con 4 balas.

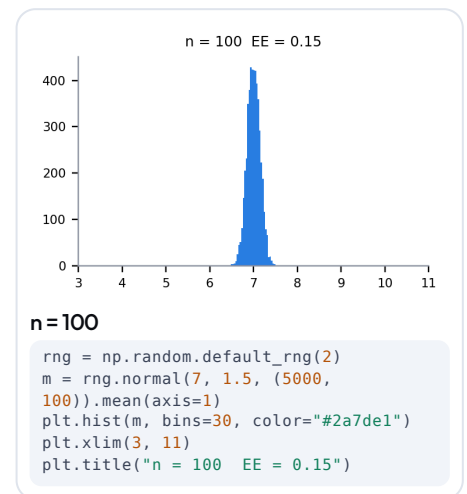
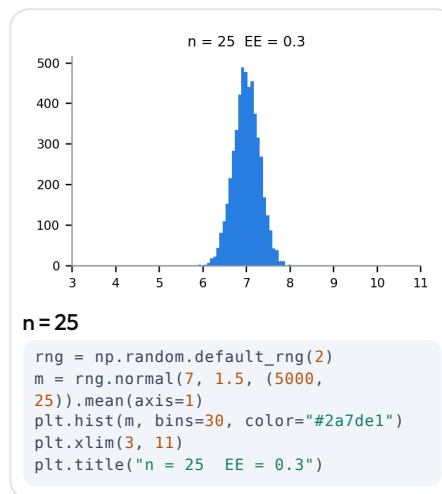
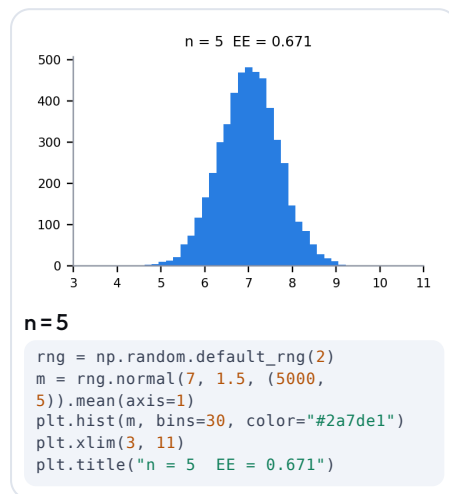
DATOS DE EJEMPLO

```
import numpy as np
from scipy import stats
rng = np.random.default_rng(2)
medias = rng.normal(7, 1.5, (5000, 25)).mean(axis=1)
```

MEDIDA	DESCRIPCIÓN	EJEMPLO	RESULTADO
$E[\bar{x}] = \mu$	El promedio de las medias es la media poblacional.	<code>medias.mean()</code>	$E[\bar{x}] = \mu$
Media simulada	Promedio de 5000 medias ($\mu = 7$).	<code>round(medias.mean(), 3)</code>	6.999
$\text{Var}(\bar{x}) = \sigma^2/n$	$\sigma^2 = 2.25$ y $n = 25$: la teoría da 0.09.	<code>round(1.5 ** 2 / 25, 3)</code>	0.09
Varianza simulada	Varianza de las 5000 medias.	<code>round(medias.var(), 3)</code>	0.087
Error estándar	$EE = \sigma/\sqrt{n} = 1.5 / \sqrt{25}$.	<code>round(1.5 / np.sqrt(25), 3)</code>	0.3
EE simulado	Desviación de las 5000 medias.	<code>round(medias.std(), 3)</code>	0.296
Cuadruplicar n	$n = 100$: el error estándar se reduce a la mitad.	<code>round(1.5 / np.sqrt(100), 3)</code>	0.15

Población: notas con $\mu = 7$ y $\sigma = 1.5$. Cada media se calcula con $n = 25$ notas.

Gráficos de ejemplo



Ley de los grandes números

Cuando n crece, el promedio de la muestra se acerca a la media real μ . Con pocos datos hay mucha variación; con muchos, el promedio se estabiliza.

RECUERDA Si lanzas un dado 5 veces puedes tener mala suerte; con 10 000 lanzamientos, el promedio casi siempre es 3.5.

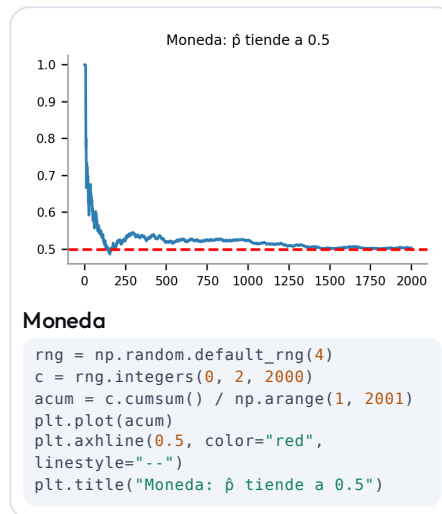
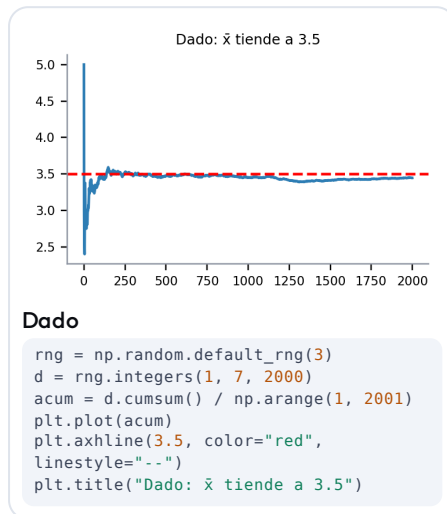
DATOS DE EJEMPLO

```
import numpy as np
from scipy import stats
rng = np.random.default_rng(3)
dado = rng.integers(1, 7, 10000)
```

MEDIDA	DESCRIPCIÓN	EJEMPLO	RESULTADO
Media del dado	$\mu = (1+2+3+4+5+6)/6 = 3.5$.	<code>np.arange(1, 7).mean()</code>	3.5
$n = 10$	Promedio de 10 lanzamientos.	<code>round(dado[:10].mean(), 3)</code>	2.9
$n = 100$	Promedio de 100 lanzamientos.	<code>round(dado[:100].mean(), 3)</code>	3.46
$n = 10\ 000$	Promedio de 10 000 lanzamientos.	<code>round(dado.mean(), 3)</code>	3.488
Distancia a μ	$ \bar{x} - 3.5 $ para $n = 10$ y $n = 10\ 000$.	<code>e1 = abs(dado[:10].mean() - 3.5)</code> <code>e2 = abs(dado.mean() - 3.5)</code> <code>print(round(e1, 3), round(e2, 3))</code>	0.6 0.012
Moneda	Proporción de caras ($p = 0.5$) con $n = 10\ 000$.	<code>moneda = rng.integers(0, 2, 10000)</code> <code>round(moneda.mean(), 4)</code>	0.4991
Promedio acumulado	Promedio tras 1, 2, 3... lanzamientos.	<code>n = np.arange(1, 10001)</code> <code>acum = dado.cumsum() / n</code> <code>i = [0, 9, 99, 999]</code> <code>print(np.round(acum[i], 3))</code>	[5. 2.9 3.46 3.453]

La ley NO dice que el azar "compense" lo pasado: solo que el promedio se estabiliza.

Gráficos de ejemplo



Teorema del Límite Central

Si n es grande ($n \geq 30$), $\bar{x}$ sigue aproximadamente una Normal ($\mu, \sigma/\sqrt{n}$) aunque la población NO sea normal. Se estandariza con $Z = (\bar{x} - \mu) / (\sigma/\sqrt{n})$.

RECUERDA Es como mezclar muchos ingredientes raros: el resultado promedio siempre tiene forma de campana.

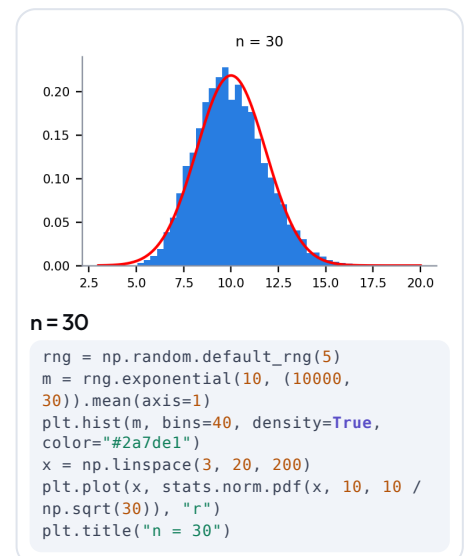
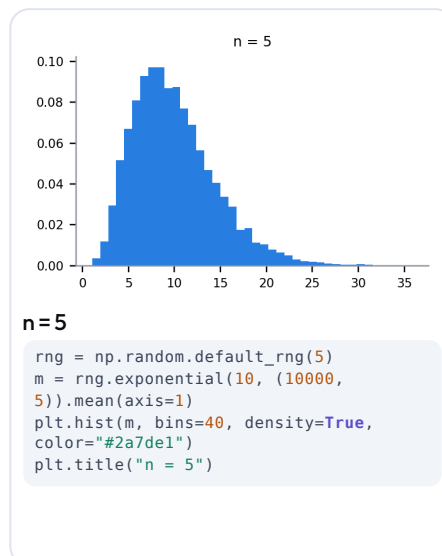
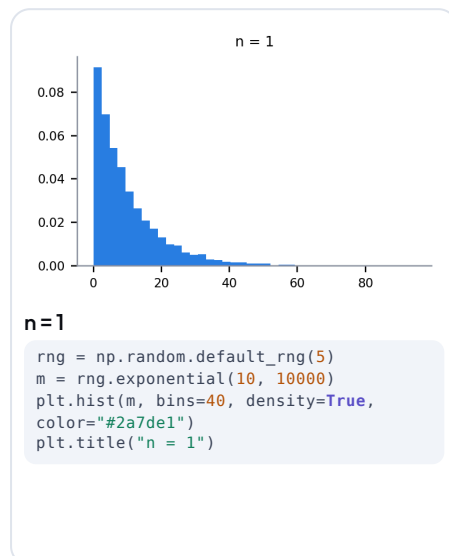
DATOS DE EJEMPLO

```
import numpy as np
from scipy import stats
rng = np.random.default_rng(5)
espera = rng.exponential(10, 10000)
medias = rng.exponential(10, (10000, 30)).mean(axis=1)
ee = 10 / np.sqrt(30)
```

PASO	DESCRIPCIÓN	EJEMPLO	RESULTADO
Población exponencial	Tiempo de espera del bus: media 10 min, muy sesgada.	<code>round(stats.skew(espera), 2)</code>	2.0
$\bar{x}$ con $n = 30$	Las medias ya son casi simétricas.	<code>round(stats.skew(medias), 2)</code>	0.36
Error estándar	$\bar{x} \sim \text{Normal}(\mu = 10, EE = \sigma/\sqrt{30} = 10/\sqrt{30})$.	<code>round(ee, 4)</code>	1.8257
Media y EE simulados	Valores de las 10 000 medias ($n = 30$).	<code>print(round(medias.mean(), 3))</code> <code>print(round(medias.std(), 3))</code>	10.003 1.817
$Z = (\bar{x} - \mu)/EE$	Si $\bar{x} = 12$ min: $Z = (12 - 10)/1.8257$.	<code>round((12 - 10) / ee, 4)</code>	1.0954
$P(\bar{x} > 12)$ teórica	Con la Normal del TLC.	<code>round(stats.norm.sf(12, 10, ee), 4)</code>	0.1367
$P(\bar{x} > 12)$ simulada	Fracción de medias mayores a 12.	<code>round((medias > 12).mean(), 4)</code>	0.1403

Para la exponencial con media 10 se cumple $\sigma = 10$.

Gráficos de ejemplo



Convergencia en distribución

Una distribución puede parecerse cada vez más a otra al crecer n . La Binomial(n, p) se acerca a Normal($np, \sqrt{npq}$) y la Poisson(λ) a Normal($\lambda, \sqrt{\lambda}$). Se usa la corrección de continuidad: sumar 0.5.

RECUERDA Es como acercar una foto pixelada: los escalones de la binomial se suavizan hasta parecer la curva normal.

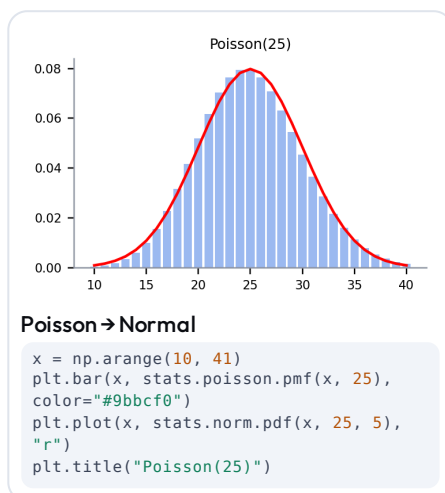
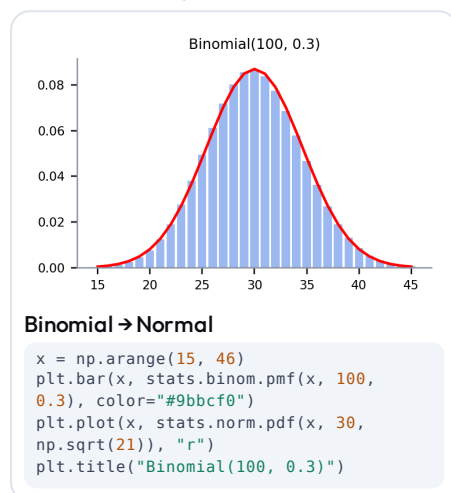
DATOS DE EJEMPLO

```
import numpy as np
from scipy import stats
sd = np.sqrt(100 * 0.3 * 0.7)
```

COMPARACIÓN	DESCRIPCIÓN	EJEMPLO	RESULTADO
Binomial exacta	$P(X \leq 35)$ con $n = 100$ y $p = 0.3$.	<code>stats.binom.cdf(35, 100, .3).round(4)</code>	0.8839
Normal aproximada	Normal(30, $\sqrt{21}$) con corrección: $P(X \leq 35.5)$.	<code>stats.norm.cdf(35.5, 30, sd).round(4)</code>	0.885
Sin corrección	Usar 35 en vez de 35.5 empeora la aproximación.	<code>stats.norm.cdf(35, 30, sd).round(4)</code>	0.8624
Poisson exacta	$P(X \leq 30)$ con $\lambda = 25$.	<code>round(stats.poisson.cdf(30, 25), 4)</code>	0.8633
Normal para Poisson	Normal(25, $\sqrt{25}$) con $P(X \leq 30.5)$.	<code>round(stats.norm.cdf(30.5, 25, 5), 4)</code>	0.8643
Error con $n = 100$	Mayor diferencia entre exacta y normal.	<pre>k = np.arange(101) ap = stats.norm.cdf((k + .5 - 30) / sd) ex = stats.binom.cdf(k, 100, 0.3) print(abs(ex - ap).max().round(4))</pre>	0.0058
Error con $n = 10$	Con pocos datos la aproximación empeora.	<pre>k = np.arange(11) ap = stats.norm.cdf((k + .5 - 3) / 1.449) ex = stats.binom.cdf(k, 10, 0.3) print(abs(ex - ap).max().round(4))</pre>	0.0178

Regla práctica para la binomial: $np \geq 5$ y $nq \geq 5$.

Gráficos de ejemplo



Distribución de la proporción muestral

La proporción de éxitos en una muestra, $\hat{p} = X/n$, se distribuye aproximadamente $\text{Normal}(p, \sqrt{(pq/n)})$, con $q = 1 - p$. Funciona si $np \geq 5$ y $nq \geq 5$.

RECUERDA Encuesta de salida: si 60 % de ESPOLusa bus, en 100 encuestados verás alrededor de 60 %, con un poco de variación.

DATOS DE EJEMPLO

```
import numpy as np
from scipy import stats
ee = np.sqrt(0.6 * 0.4 / 100)
```

MEDIDA	DESCRIPCIÓN	EJEMPLO	RESULTADO
Error estándar	$EE = \sqrt{(pq/n)}$ con $p = 0.6$ y $n = 100$.	<code>round(np.sqrt(0.6 * 0.4 / 100), 4)</code>	0.049
Condición	np y nq deben ser al menos 5.	<code>print(100 * 0.6, 100 * 0.4)</code>	60.0 40.0
$P(\hat{p} > 0.7)$	Prob. de que más de 70 % usen bus.	<code>round(stats.norm.sf(0.7, 0.6, ee), 4)</code>	0.0206
$P(0.55 < \hat{p} < 0.65)$	Prob. de caer a ± 5 puntos de p .	<code>a = stats.norm.cdf(0.65, 0.6, ee)</code> <code>b = stats.norm.cdf(0.55, 0.6, ee)</code> <code>round(a - b, 4)</code>	0.6926
Z de $\hat{p} = 0.7$	$Z = (\hat{p} - p) / EE$.	<code>round((0.7 - 0.6) / ee, 4)</code>	2.0412
Binomial exacta	$P(\hat{p} > 0.7) = P(X \geq 71)$ con $n = 100$.	<code>stats.binom.sf(70, 100, 0.6).round(4)</code>	0.0148
Simulación	Fracción de 10 000 encuestas con $\hat{p} > 0.7$.	<code>rng = np.random.default_rng(6)</code> <code>x = rng.binomial(100, 0.6, 10000)</code> <code>ph = x / 100</code> <code>round((ph > 0.7).mean(), 4)</code>	0.0146

Aquí $p = 0.6$ y $n = 100$. $\hat{p}$ solo toma valores discretos, por eso la Normal (0.0206) se aleja un poco de la exacta (0.0148).

Distribución de la diferencia de medias

Con dos muestras independientes, $\bar{x}_1 - \bar{x}_2$ tiene media $\mu_1 - \mu_2$ y varianza $\sigma_1^2/n_1 + \sigma_2^2/n_2$. Si las poblaciones son normales o los n son grandes, es Normal.

RECUERDA Las varianzas SE SUMAN aunque las medias se resten: comparar dos grupos agrega incertidumbre de ambos.

DATOS DE EJEMPLO

```
import numpy as np
from scipy import stats
ee = np.sqrt(2.25 / 40 + 3.24 / 50)
```

MEDIDA	DESCRIPCIÓN	EJEMPLO	RESULTADO
Media de la diferencia	$\mu_1 - \mu_2 = 7.2 - 6.8$.	<code>round(7.2 - 6.8, 2)</code>	0.4
Varianza	$\sigma_1^2/n_1 + \sigma_2^2/n_2 = 2.25/40 + 3.24/50$.	<code>round(2.25 / 40 + 3.24 / 50, 5)</code>	0.12105
Error estándar	Raíz de la varianza.	<code>round(ee, 4)</code>	0.3479
$P(\bar{x}_1 - \bar{x}_2 > 0)$	Prob. de que la carrera A saque más media.	<code>round(stats.norm.sf(0, 0.4, ee), 4)</code>	0.8749
$P(\text{diferencia} > 0.8)$	Prob. de una diferencia grande.	<code>round(stats.norm.sf(0.8, 0.4, ee), 4)</code>	0.1251

MEDIDA	DESCRIPCIÓN	EJEMPLO	RESULTADO
Z	$Z = ((\bar{x}_1 - \bar{x}_2) - (\mu_1 - \mu_2)) / EE$, con 0.8.	<code>round((0.8 - 0.4) / ee, 4)</code>	1.1497
Simulación	Fracción de 10 000 pares con diferencia > 0.	<pre>rng = np.random.default_rng(7) a = rng.normal(7.2, 1.5, (10000, 40)) b = rng.normal(6.8, 1.8, (10000, 50)) d = a.mean(axis=1) - b.mean(axis=1) round((d > 0).mean(), 4)</pre>	0.8762

Carrera A: $\mu = 7.2$, $\sigma = 1.5$, $n = 40$. Carrera B: $\mu = 6.8$, $\sigma = 1.8$, $n = 50$.

TEMA 66

Distribución chi-cuadrado

Si $Z_1, \dots, Z_k$ son normales estándar independientes, la suma $Z_1^2 + \dots + Z_k^2$ sigue una chi-cuadrado con k grados de libertad (gl). Solo toma valores positivos y es asimétrica.

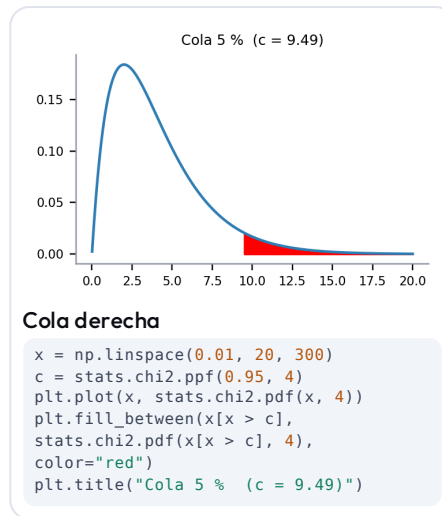
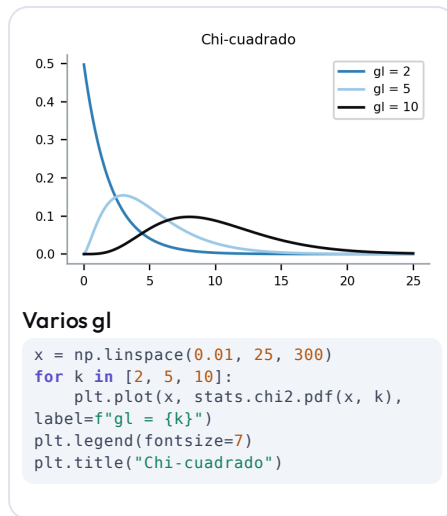
RECUERDA Chi-cuadrado = sumar cuadrados de errores: nunca es negativa, y mientras más gl, más a la derecha y más simétrica.

DATOS DE EJEMPLO

```
import numpy as np
from scipy import stats
rng = np.random.default_rng(8)
```

MEDIDA	DESCRIPCIÓN	EJEMPLO	RESULTADO
Suma de Z^2	4 normales estándar al cuadrado, sumadas.	<pre>z = rng.standard_normal((10000, 4)) q = (z ** 2).sum(axis=1) print(q[:3].round(3))</pre>	[6.784 7.099 3.446]
Media = k	Para $k = 4$ gl: simulada y teórica.	<pre>z = rng.standard_normal((10000, 4)) q = (z ** 2).sum(axis=1) print(round(q.mean(), 3), 4)</pre>	3.958 4
Varianza = 2k	Para $k = 4$: simulada y teórica.	<pre>z = rng.standard_normal((10000, 4)) q = (z ** 2).sum(axis=1) print(round(q.var(), 3), 8)</pre>	7.892 8
chi2.cdf()	$P(x^2 \leq 9.4877)$ con 4 gl.	<code>round(stats.chi2.cdf(9.4877, 4), 4)</code>	0.95
chi2.sf()	$P(x^2 > 7.8147)$ con 3 gl.	<code>round(stats.chi2.sf(7.8147, 3), 4)</code>	0.05
chi2.ppf()	Valor crítico: el 95 % queda a su izquierda (4 gl).	<code>round(stats.chi2.ppf(0.95, 4), 4)</code>	9.4877
Más gl	El valor crítico crece con los gl.	<pre>gl = [1, 5, 10] q = stats.chi2.ppf(0.95, gl) print(q.round(3))</pre>	[3.841 11.07 18.307]

Gráficos de ejemplo



TEMA 67

Distribución de la varianza muestral

Si la población es normal con varianza σ^2 , entonces $(n-1)S^2/\sigma^2$ sigue una chi-cuadrado con $n-1$ grados de libertad. Sirve para calcular probabilidades sobre S^2 .

RECUERDA Cada muestra tiene su propia varianza S^2 ; la chi-cuadrado dice qué tan lejos de σ^2 puede caer por pura suerte.

DATOS DE EJEMPLO

```
import numpy as np
from scipy import stats
rng = np.random.default_rng(9)
x = rng.normal(70, 2, (10000, 10))
s2 = x.var(axis=1, ddof=1)
```

MEDIDA	DESCRIPCIÓN	EJEMPLO	RESULTADO
Estadístico χ^2	Con $n = 10$, $\sigma^2 = 4$ y $S^2 = 6:9:6/4$.	<code>round((10 - 1) * 6 / 4, 2)</code>	13.5
$P(S^2 > 6)$	Prob. de que S^2 supere 6 (χ^2 con 9 gl > 13.5).	<code>round(stats.chi2.sf(13.5, 9), 4)</code>	0.1413
$P(S^2 < 2)$	$\chi^2 < 9:2/4 = 4.5$ con 9 gl.	<code>round(stats.chi2.cdf(4.5, 9), 4)</code>	0.1245
$E[S^2] = \sigma^2$	La media de S^2 es la varianza real ($\sigma^2 = 4$).	<code>round(s2.mean(), 3)</code>	4.029
$\text{Var}(S^2)$	$2\sigma^4/(n-1) = 2:16/9$.	<code>round(2 * 16 / 9, 3)</code>	3.556
$\text{Var}(S^2)$ simulada	Varianza de las 10 000 S^2 .	<code>round(s2.var(), 3)</code>	3.629
Simulación	Fracción de S^2 mayores a 6.	<code>round((s2 > 6).mean(), 4)</code>	0.149

Notas con $\sigma = 2$ ($\sigma^2 = 4$) y muestras de $n = 10$.

Distribución t de Student

Cuando σ es desconocida se reemplaza por S : $T = (\bar{x} - \mu) / (S/\sqrt{n})$ sigue una t con $n-1$ gl. Se parece a la normal, pero con colas más pesadas, sobre todo con n pequeño.

RECUERDA La t es una normal «con miedo»: como solo tienes S y no σ , deja más probabilidad en los extremos.

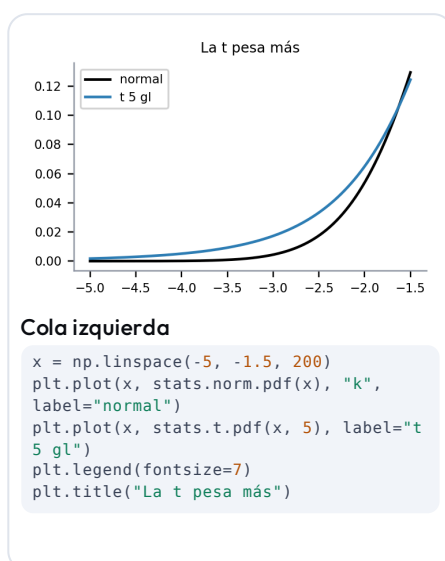
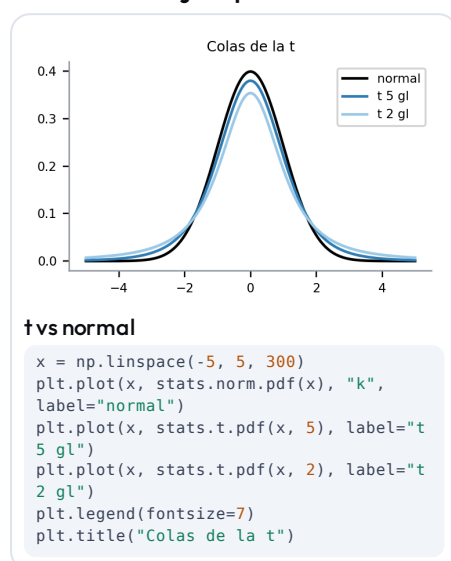
DATOS DE EJEMPLO

```
import numpy as np
from scipy import stats
rng = np.random.default_rng(10)
```

MEDIDA	DESCRIPCIÓN	EJEMPLO	RESULTADO
t.ppf()	Valor crítico 95 % con $n = 10$ (9 gl).	<code>round(stats.t.ppf(0.975, 9), 4)</code>	2.2622
norm.ppf()	El mismo valor crítico en la normal.	<code>round(stats.norm.ppf(0.975), 4)</code>	1.96
Más datos	Con 29 y 120 gl la t se acerca a 1.96.	<code>q = stats.t.ppf(0.975, [29, 120])</code> <code>print(q.round(4))</code>	[2.0452 1.9799]
Colas pesadas	$P(T > 2)$ con 5 gl.	<code>round(stats.t.sf(2, 5), 4)</code>	0.051
Cola normal	$P(Z > 2)$: menos probabilidad en la cola.	<code>round(stats.norm.sf(2), 4)</code>	0.0228
Varianza	gl/(gl-2) con 5 gl: mayor que 1.	<code>round(5 / (5 - 2), 4)</code>	1.6667
T simulado	Fracción de $T > 2$ con $n = 6$ notas.	<code>x = rng.normal(7, 1.5, (20000, 6))</code> <code>s = x.std(axis=1, ddof=1)</code> <code>m = x.mean(axis=1)</code> <code>t = (m - 7) / (s / 6 ** 0.5)</code> <code>round((t > 2).mean(), 3)</code>	0.048

La simulación con $n = 6$ (5 gl) da un valor cercano a $P(T > 2) = 0.051$.

Gráficos de ejemplo



Distribución F de Fisher

Si dos poblaciones normales tienen varianzas σ_1^2 y σ_2^2 , el cociente $(S_1^2/\sigma_1^2) / (S_2^2/\sigma_2^2)$ sigue una F con n_1-1 y n_2-1 grados de libertad. Se usa para comparar varianzas.

RECUERDA F compara dos varianzas dividiéndolas: si el cociente es cercano a 1, los grupos varían parecido.

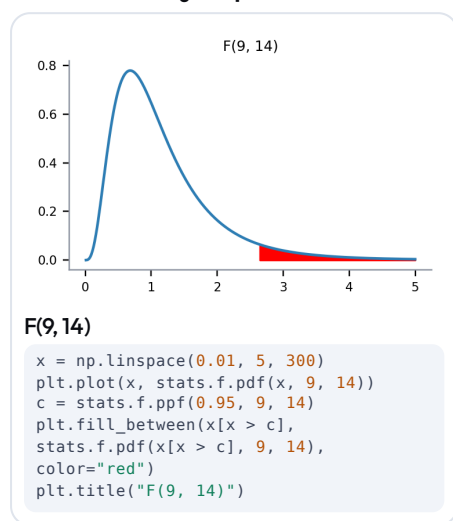
DATOS DE EJEMPLO

```
import numpy as np
from scipy import stats
rng = np.random.default_rng(11)
```

MEDIDA	DESCRIPCIÓN	EJEMPLO	RESULTADO
Cociente F	$S_1^2 = 4$ ($n = 10$) y $S_2^2 = 2.5$ ($n = 15$), σ iguales.	<code>round(4 / 2.5, 4)</code>	1.6
f.sf()	$P(F > 1.6)$ con 9 y 14 gl.	<code>round(stats.f.sf(1.6, 9, 14), 4)</code>	0.2076
f.cdf()	$P(F \leq 1.6)$ con 9 y 14 gl.	<code>round(stats.f.cdf(1.6, 9, 14), 4)</code>	0.7924
f.ppf()	Valor crítico: 95 % a la izquierda (9 y 14 gl).	<code>round(stats.f.ppf(0.95, 9, 14), 4)</code>	2.6458
Cola izquierda	Se obtiene con $1/F$ y los gl al revés.	<code>a = stats.f.ppf(0.05, 9, 14)</code> <code>b = 1 / stats.f.ppf(0.95, 14, 9)</code> <code>print(round(a, 4), round(b, 4))</code>	0.3305 0.3305
Media de F	$d_2/(d_2 - 2)$ con $d_2 = 14$.	<code>round(14 / (14 - 2), 4)</code>	1.1667
Simulación	Fracción de $F > 2.6458$ con 9 y 14 gl.	<code>a = rng.normal(0, 2, (20000, 10))</code> <code>b = rng.normal(0, 2, (20000, 15))</code> <code>va = a.var(axis=1, ddof=1)</code> <code>vb = b.var(axis=1, ddof=1)</code> <code>round(((va / vb) > 2.6458).mean(), 3)</code>	0.051

Si $\sigma_1 = \sigma_2$, $F = S_1^2/S_2^2$. En la simulación, 2.6458 es el valor crítico del 95 %.

Gráficos de ejemplo



Estimadores y sesgo

Un estimador puntual es una fórmula con la muestra que da un valor para el parámetro. Es insesgado si en promedio acierta: $E[\hat{\theta}] = \theta$. $\bar{x}$ y S^2 (con $n-1$) son insesgados; dividir entre n subestima la varianza.

RECUERDA Una balanza que siempre marca 100 g de menos tiene sesgo: por más pesadas que hagas, el error no se va.

DATOS DE EJEMPLO

```
import numpy as np
from scipy import stats
rng = np.random.default_rng(12)
x = rng.normal(70, 2, (20000, 5))
```

ESTIMADOR	DESCRIPCIÓN	EJEMPLO	RESULTADO
Sesgo	$\text{sesgo} = E[\text{estimador}] - \text{parámetro}$.	$\text{sesgo} = \text{media} - \text{parametro}$	Insesgado si $\text{sesgo} = 0$
$\bar{x}$ es insesgado	Promedio de 20 000 medias ($\mu = 70$).	<code>round(x.mean(axis=1).mean(), 3)</code>	70.001
S^2 con <code>ddof=1</code>	Divide entre $n-1$ ($\sigma^2 = 4$).	<code>round(x.var(axis=1, ddof=1).mean(), 3)</code>	3.979
Varianza con <code>ddof=0</code>	Divide entre n : sesgada.	<code>round(x.var(axis=1, ddof=0).mean(), 3)</code>	3.183
Sesgo de <code>ddof=0</code>	Teoría: $E = \sigma^2(n-1)/n = 4 \cdot 4/5$.	<code>round(4 * 4 / 5, 3)</code>	3.2
Sesgo de S	S no es insesgado para $\sigma = 2$.	<code>round(x.std(axis=1, ddof=1).mean(), 3)</code>	1.875
Mediana	También es insesgada si la población es simétrica.	<code>round(np.median(x, axis=1).mean(), 3)</code>	70.0

Población normal con $\mu = 70$ y $\sigma = 2$; muestras de $n = 5$.

Intervalo de confianza para la media

Es un rango que probablemente contiene a μ . Con σ conocida: $\bar{x} \pm z \cdot \sigma / \sqrt{n}$. Con σ desconocida: $\bar{x} \pm t \cdot S / \sqrt{n}$ con $n-1$ gl. El término que se suma y resta es el margen de error.

RECUERDA Dices «el promedio está entre 6.9 y 7.9» en vez de un solo número: es más honesto, porque la muestra nunca es perfecta.

DATOS DE EJEMPLO

```
import numpy as np
from scipy import stats
notas = np.array([6.5, 7.0, 8.2, 5.9, 7.8, 6.1, 9.0, 7.4, 6.8, 8.0, 7.2, 5.6])
ee = 1.5 / 6
```

CÁLCULO	DESCRIPCIÓN	EJEMPLO	RESULTADO
Con σ conocida	$n = 36$, $\bar{x} = 7.4$, $\sigma = 1.5$: $z = 1.96$, $\text{margen} = 0.49$.	<pre>m = 1.96 * 1.5 / 6 print(round(m, 3)) print(round(7.4 - m, 3)) print(round(7.4 + m, 3))</pre>	0.49 6.91 7.89
<code>stats.norm.interval()</code>	El mismo intervalo al 95 %.	<pre>ci = stats.norm.interval(.95, 7.4, ee) print(np.round(ci, 3))</pre>	[6.91 7.89]
Al 99 %	Más confianza = intervalo más ancho.	<pre>ci = stats.norm.interval(.99, 7.4, ee) print(np.round(ci, 3))</pre>	[6.756 8.044]

CÁLCULO	DESCRIPCIÓN	EJEMPLO	RESULTADO
Con σ desconocida	12 notas: usa S y la t con 11 gl.	<pre>print(notas.mean().round(3)) s = notas.std(ddof=1) print(s.round(3)) t = stats.t.ppf(0.975, 11) print(t.round(4))</pre>	7.125 1.016 2.201
<code>stats.t.interval()</code>	Intervalo al 95 % con las 12 notas.	<pre>m = notas.mean() se = stats.sem(notas) ci = stats.t.interval(.95, 11, m, se) print(np.round(ci, 3))</pre>	[6.479 7.771]
Tamaño de muestra	Para margen 0.25: $n = (z \cdot \sigma / E)^2$.	<pre>n = (1.96 * 1.5 / 0.25) ** 2 print(round(n, 2), int(np.ceil(n)))</pre>	138.3 139
Cobertura	De 1000 intervalos al 95 %, ¿cuántos atrapan $\mu = 7$?	<pre>rng = np.random.default_rng(13) x = rng.normal(7, 1.5, (1000, 36)) m = x.mean(axis=1) ok = abs(m - 7) <= 1.96 * ee print(ok.mean())</pre>	0.943

Interpretación: el 95 % es la confianza en el MÉTODO, no la probabilidad de un intervalo ya calculado.